# Projet master

YAHYAOUI Ismail

Encadrant : GALERNE Bruno

2022/2023

Sujet : Algorithme de descente de gradient avec moment (Heavy-ball algorithm) : Théorie et application à la restauration d'images

Université d'Orléans

Contexte :

L'algorithme de descente de gradient est une méthode couramment utilisée pour résoudre des problèmes d'optimisation dans de nombreux domaines, y compris le traitement d'images. Cependant, dans certaines situations, cet algorithme peut être relativement lent et nécessiter un grand nombre d'itérations pour converger vers la solution optimale.

Pour remédier à ce problème, des améliorations ont été proposées, et l'une d'entre elles est l'algorithme de descente de gradient avec moment, également connu sous le nom d'algorithme de la boule lourde (Heavy-ball algorithm). Ce dernier consiste à ajouter un terme de moment à l'algorithme de descente de gradient traditionnel.

Le moment, dans ce contexte, est une mémoire à court terme de la direction des itérations précédentes. Il permet de conserver une trace des mouvements passés et de les utiliser pour influencer les mouvements futurs. En introduisant ce terme de moment, l'algorithme de descente de gradient avec moment peut accélérer la convergence et souvent atteindre la solution optimale plus rapidement que l'algorithme de descente de gradient classique.

Dans ce projet, nous nous intéresserons à la théorie derrière l'algorithme de descente de gradient avec moment, en détaillant les preuves qui démontrent l'accélération obtenue par l'ajout de ce terme de moment. Nous étudierons les conditions sous lesquelles cet algorithme est convergent et nous explorerons les propriétés mathématiques qui le caractérisent.

De plus, nous appliquerons cet algorithme d'optimisation à un domaine spécifique, à savoir la restauration d'images. La restauration d'images est un problème important dans le traitement d'images, qui vise à récupérer une image endommagée ou dégradée en utilisant des techniques de traitement du signal et des méthodes d'optimisation. Nous verrons comment l'algorithme de descente de gradient avec moment peut être adapté et appliqué pour résoudre des problèmes de restauration d'images, en exploitant ses avantages en termes de vitesse de convergence et d'efficacité.

Ce projet combinera donc une étude théorique approfondie de l'algorithme de descente de gradient avec moment, en mettant l'accent sur les preuves de convergence et d'accélération, avec une application concrète dans le domaine de la restauration d'images

# Plan


## 1 Introduction

### 1.1 Problème d'optimisation

### 1.2 Existence des solutions

### 1.3 Algorithme de descente de gradient

### 1.4 Algorithme de descente de gradient avec moment

## 2 Convergence dans le cas simple des fonctionnelles quadratiques définies positives

## 3 Convergence pour une fonction convexe à gradient Lipschitz

## 4 Convergence pour une fonction fortement convexe

## 5 Comparaison numérique avec la descente de gradient standard

## 6 Comparaison numérique avec la descente de gradient standard sur un problème de restauration d'images : Déflouage par régularisation de Tichonov (en python)

# 1 Introduction

## 1.1 Problème d'optimisation

Le domaine de l'optimisation joue un rôle essentiel dans de nombreux problèmes de la vie réelle, qu'ils relèvent des sciences, de l'ingénierie, de l'économie ou d'autres domaines. Un problème d'optimisation général peut être formulé comme suit :

$$\text{Trouver } x^* \text{ tel que } x^* \in U \text{ et } f(x^*) = \min_{x \in \mathbb{R}^n} f(x),$$

où $U$ est un ensemble donné de $\mathbb{R}^n$ et $f : \mathbb{R}^n \to \mathbb{R}$ est une fonction objectif. L'objectif est de trouver un vecteur $x^*$ qui minimise la fonction $f$ dans l'ensemble $U$. Selon les propriétés de $U$ et $f$, le problème d'optimisation peut être sans contraintes ou sous contraintes, convexe ou non convexe.

Dans ce projet, nous nous concentrons sur la résolution de problèmes d'optimisation convexes sans contraintes de dimension finie. L'optimisation convexe offre de nombreuses propriétés intéressantes, telles que la garantie d'existence et d'unicité des solutions optimales, ainsi que des méthodes efficaces pour les trouver.

L'objectif de ce projet est de présenter l'algorithme de descente de gradient avec moment, également connu sous le nom d'algorithme de la boule lourde (Heavy-ball algorithm), et d'étudier ses propriétés théoriques ainsi que son application à la restauration d'images. Cet algorithme est une extension de l'algorithme de descente de gradient traditionnel qui ajoute un terme de moment pour accélérer la convergence.

Nous commencerons par examiner les conditions d'existence et d'unicité des solutions pour les problèmes d'optimisation convexes sans contraintes. Ensuite, nous présenterons en détail l'algorithme de descente de gradient avec moment, en expliquant sa formulation mathématique et son fonctionnement. Nous étudierons les preuves théoriques qui démontrent l'accélération obtenue par l'ajout du terme de moment.

Enfin, nous appliquerons cet algorithme à un domaine spécifique, la restauration d'images. La restauration d'images est un problème crucial dans le domaine du traitement d'images, qui vise à récupérer une image endommagée ou dégradée en utilisant des techniques de traitement du signal et des méthodes d'optimisation. Nous montrerons comment l'algorithme de descente de gradient avec moment peut être adapté et appliqué pour résoudre des problèmes de restauration d'images, en exploitant ses avantages en termes de vitesse de convergence et d'efficacité.

En résumé, ce projet combine une étude théorique approfondie de l'algorithme de descente de gradient avec moment, en mettant l'accent sur les preuves de convergence et d'accélération, avec une application concrète dans le domaine de la restauration d'images. Nous explorerons ainsi les fondements théoriques de l'algorithme et son utilisation pratique pour résoudre des problèmes d'optimisation et améliorer la qualité des images restaurées.

## 1.2 Existence des solutions

L'une des questions fondamentales lorsqu'on aborde un problème d'optimisation est de déterminer l'existence d'une solution.

Considérons une fonction $f : U \subset \mathbb{R}^n \to \mathbb{R}$ continue définie sur un sous-ensemble $U$ de $\mathbb{R}^n$. Pour garantir l'existence d'une solution à notre problème d'optimisation, nous devons examiner les propriétés de la fonction $f$. Si $U$ est un ensemble compact (c'est-à-dire fermé et borné), alors la fonction $f$ est bornée sur $U$ et atteint ses bornes sur cet ensemble. Par conséquent, elle possède au moins un minimum global $x^* \in U$.

Cependant, si $U$ est non borné, la notion de coercivité peut être utilisée pour étendre ce raisonnement. Une fonction $f : \mathbb{R}^n \to \mathbb{R}$ est dite coercive si elle tend vers l'infini lorsque la norme de son argument tend vers l'infini.

En utilisant le concept de coercivité, on peut montrer que si $f$ est une fonction continue et coercive définie sur un sous-ensemble non borné de $\mathbb{R}^n$, alors il existe au moins un élément $x^*$ dans cet ensemble tel que $f(x^*)$ soit le minimum de $f$ sur cet ensemble.

Il est également important de caractériser les minima locaux d'une fonction différentiable. Si $x$ est un point où la fonction $f$ atteint un extremum local et si $f$ est différentiable en ce point, alors le gradient de $f$ en ce point est nul ($\nabla f(x) = 0$). Ces points où le gradient est nul sont appelés des points critiques de la fonction $f$. L'information fournie par la dérivée seconde de $f$ peut souvent aider à déterminer la nature des points critiques. Par exemple, si la matrice hessienne de $f$ évaluée en un point critique $x$ est positive définie, alors $x$ correspond à un minimum local strict de la fonction $f$. En général, si la matrice hessienne a des valeurs propres de signes opposés, le point critique $x$ n'est ni un minimum local ni un maximum local, mais plutôt un point selle.

Dans le cadre de notre projet, nous supposerons que la fonction $f$ est convexe. Une fonction $f : U \subset \mathbb{R}^n \to \mathbb{R}$ est dite convexe si, pour tout $x, y \in U$ et pour tout $\theta$ appartenant à l'intervalle $[0, 1]$, l'inégalité suivante est vérifiée :

$$f(\theta x + (1 - \theta)y) \leq \theta f(x) + (1 - \theta)f(y)$$

La convexité joue un rôle crucial en optimisation, car elle garantit qu'un minimum local est également un minimum global.

Ainsi, dans le contexte de l'optimisation convexe, nous pouvons utiliser la convexité de la fonction $f$ et l'existence de solutions locales pour garantir l'existence de solutions globales à notre problème d'optimisation.

En résumé, nous avons abordé l'existence des solutions dans le contexte de l'optimisation. Nous avons discuté des conditions d'existence de solutions et de la notion de convexité, qui joue un rôle crucial pour garantir l'existence de solutions globales dans les problèmes d'optimisation.

Maintenant introduisant les deux algorithmes.

## 1.3 Algorithme de descente de gradient

**Algorithme :** Algorithme de descente de gradient

**Données :** Un point initial $x^{(0)} \in \mathbb{R}^n$, un seuil de tolérance $\varepsilon > 0$
**Résultat :** Un point $x \in \mathbb{R}^n$ proche de $x^*$
**Initialiser** $x$ **:** $x \leftarrow x^{(0)}$ ; $k \leftarrow 0$ ;
**Tant que** $||\nabla f(x)|| > \varepsilon$ **faire**

1. Calculer $d^{(k)} = -\nabla f(x)$ (ou $d^{(k)} = -\nabla f(x^{(k)})$).
2. Déterminer un pas de descente $t^{(k)} > 0$ par la méthode exacte (ou par la méthode de rebroussement).
3. Mettre à jour $x$ : $x \leftarrow x^{(k+1)} = x^{(k)} + t^{(k)} d^{(k)}$ ; $k \leftarrow k + 1$ ;

**fin**

Lorsque nous allons comparer ces deux algorithmes nous allons prendre un pas t qui est fixe.

En étudiant un algorithme de descente, nous nous intéressons à sa convergence, c'est-à-dire à la question de savoir si la suite de points $(x^{(k)})_{k\in\mathbb{N}}$ qu'il génère converge vers la solution optimale $x^\star$.

Dans le cas d'une fonction objectif fortement convexe, il est possible d'établir une relation entre la norme de la différence entre le point courant et la solution optimale $\|x - x^\star\|$ et la norme du gradient de la fonction $\|\nabla f(x)\|$. Ainsi, si le critère d'arrêt $\|\nabla f(x)\| \leq \varepsilon$ est satisfait, nous pouvons conclure que $\|x - x^\star\| \leq \frac{2\varepsilon}{m}$, où $m$ est une constante liée à la forte convexité de la fonction.

De plus, il est intéressant de considérer la convergence de la suite $(f(x^{(k)}) - p^\star)_{k\in\mathbb{N}}$ qui représente la différence entre la valeur de la fonction objectif à chaque itération et la valeur optimale $p^\star$. Cette convergence pour la fonction objectif permet d'évaluer la convergence globale de l'algorithme et de mesurer la proximité de la solution trouvée avec la solution optimale.

En résumé, l'étude de la convergence d'un algorithme de descente est essentielle pour évaluer son efficacité et sa capacité à atteindre la solution optimale. Elle repose sur des inégalités liées à la forte convexité de la fonction et fournit des informations précieuses sur la proximité de la solution

obtenue par rapport à la solution optimale.

**Théorème :** Soient $f : \mathbb{R}^n \to \mathbb{R}$ une fonction fortement convexe telle que

$$\forall x, h \in \mathbb{R}^n, m\|h\|^2 \leq \langle \nabla^2 f(x)h, h\rangle \leq M\|h\|^2$$

avec $0 < m \leq M$, et soit $x(0)$ un point quelconque de $\mathbb{R}^n$. Alors l'algorithme de descente de gradient converge et la suite $(f(x(k)) - p^\star)_{k\in\mathbb{N}}$ converge linéairement, où $p^\star$ est la valeur optimale :

$$k \in \mathbb{N},\ f(x(k)) - p^\star \leq c^k(f(x^{(0)}) - p^\star),$$

où $c \in [0, 1[$ dépend de la méthode de recherche du pas de descente. La valeur de $c$ est donnée par :

- $c = 1 - \frac{m}{M}$ pour la méthode exacte/optimale,
- $c = 1 - \min\left\{2m\alpha, 2\beta\alpha\frac{m}{M}\right\}$ pour la méthode de rebroussement utilisant les constantes $\alpha \in ]0, \frac{1}{2}[$ et $\beta \in ]0, 1[$.

## 1.4 Algorithme de descente de gradient avec moment

**Algorithme :** Algorithme de descente de gradient avec moment

**Données :** Un point initial $x^{(0)} \in \mathbb{R}^n$, un seuil de tolérance $\varepsilon > 0$, une pas de descente $\alpha > 0$ et un moment $\beta > 0$

**Résultat :** Un point $x \in \mathbb{R}^n$ proche de $x^*$

**Initialiser** $x$ **:** $x \leftarrow x^{(0)}$ ; $k \leftarrow 0$ ;

**Tant que** $||\nabla f(x)|| > \varepsilon$ **faire**

1. Mettre à jour $x$ : $x \leftarrow x^{(k+1)} = x^{(k)} - \alpha\nabla f(x^k) + \beta(x^k - x^{k-1})$;k $\leftarrow k + 1$ ;

**fin**

Cet algorithme de descente de gradient avec moment que nous allons étudier par la suite converge beaucoup plus vite que l'algorithme classique.

Nous allons tout d'abord étudier la convergence dans le cas simple des fonctionnelles quadratiques définie positives ensuite pour les fonctions convexes à gradient Lipschitz et pour les fonctions fortement convexes. À la fin nous allons faire la comparaison numérique des deux algorithmes.

## 2 Convergence dans le cas simple des fonctionnelles quadratiques définie positives

Dans cette partie, nous allons étudier la convergence de l'algorithme de descente de gradient avec moment (Heavy-ball algorithm) dans le cas simple des fonctionnelles quadratiques définie positives.

On rappele que l'itération de Heavy-ball est

$$x_{k+1} = x_k - \alpha \nabla f(x_k) + \beta(x_k - x_{k-1}),$$

où $\alpha > 0$ est le pas de descente et $\beta > 0$ le moment.

**Définition :** On appele fonctionnelle quadratique toute fonction $f : \mathbb{R}^n \to \mathbb{R}$ de la forme $f(x) = \frac{1}{2}\langle Ax, x\rangle - \langle b, x\rangle + c$, où $A \in M_n(\mathbb{R})$ est une matrice carrée symétrique, $b \in \mathbb{R}^n$ et $c \in \mathbb{R}$. La proposition suivante résume les propriétés des fonctionnelles quadratiques.

**Proposition :** Soit $f : \mathbb{R}^n \to \mathbb{R}$ une fonctionnelle quadratique de la forme $f(x) = \frac{1}{2}\langle Ax, x\rangle - \langle b, x\rangle + c$, avec $A \in M_n(\mathbb{R})$ symétrique. Alors :
(a) $f$ est $C^2$ sur $\mathbb{R}^n$ (et même $C^\infty$).
(b) Pour tout $x \in \mathbb{R}^n$, $\nabla f(x) = Ax - b$ et $\nabla^2 f(x) = A$.
(c) $f$ est convexe si et seulement si $A$ est positive.
(d) $f$ est strictement convexe si et seulement si $A$ est définie positive.
(e) $f$ est coercive si et seulement si $A$ est définie positive.
(f) $\inf_{x\in\mathbb{R}^n} f(x)$ est fini si et seulement si $A$ est positive et si le système linéaire $Ax = b$ admet au moins une solution. De plus, l'ensemble des solutions de $Ax = b$ est l'ensemble des minima globaux de $f$. Ainsi, résoudre le problème d'optimisation associé à $f$ revient à résoudre le système linéaire $Ax = b$.

**Proposition :** Si la matrice $A$ est positive définie et symétrique, alors :
(a) Toutes les valeurs propres de $A$ sont positives.
(b)$A$ est inversible et une solution optimale $x^*$ existe, où $x^* = A^{-1}b$.

**Proposition :** Soit $M$ une matrice symétrique réelle d'ordre $n$. Elle est dite définie positive si elle est positive et inversible, autrement dit si elle vérifie l'une des quatre propriétés équivalentes suivantes :
(a) Pour toute matrice colonne non nulle $\mathbf{x}$ à $n$ éléments réels, on a $\mathbf{x}^\mathsf{T} M\mathbf{x} > 0$. Autrement dit, la forme quadratique définie par $M$ est strictement positive pour $\mathbf{x} \neq \mathbf{0}$.
(b) Toutes les valeurs propres de $M$ (qui sont nécessairement réelles) sont strictement positives.
(c) La forme bilinéaire symétrique $\mathbb{R}^n \to \mathbb{R}$, $(\mathbf{x}, \mathbf{y}) \mapsto \mathbf{x}^\mathsf{T} M\mathbf{y}$, est un produit scalaire sur $\mathbb{R}^n$.
(d) Il existe une matrice $N \in \mathbf{M}_n(\mathbb{R})$ inversible telle que $M = N^\mathsf{T} N$ (autrement dit, $M$ est congruente à la matrice identité).

Avant de montrer la convergence on doit voir trois lemmes. Le premier lemme sera utile pour montrer le lemme 2.1 et le lemme 2.2 et 2.3 seront utile pour montrer la convergence.

**Lemme 2.1 :**

Soit $A \in \mathbf{M}_n(\mathbb{C})$ avec un rayon spectral $\rho(A)$. Alors $\rho(A) < 1$ si et seulement si

$$\lim_{k\to\infty} A^k = 0.$$

De plus, si $\rho(A) > 1$, alors

$$\lim_{k\to\infty} \|A^k\| = \infty.$$

**Preuve :**

Supposons que $\lim_{k\to\infty} A^k = 0$. Nous allons montrer que $\rho(A) < 1$. Soit $(v, \lambda)$ une paire vecteur propre, valeur propre pour $A$. Comme $A^k v = \lambda^k v$, on a

$$0 = \left(\lim_{k\to\infty} A^k\right) v = \lim_{k\to\infty} \left(A^k v\right) = \lim_{k\to\infty} (\lambda^k v) = v \lim_{k\to\infty} \lambda^k.$$

Comme $v \neq 0$ ,on a

$$\lim_{k\to\infty} \lambda^k = 0,$$

donc $|\lambda| < 1$. Comme c'est vrai pour toute valeur propre $\lambda$, on a alors que $\rho(A) < 1$. Supposons que $\rho(A) < 1$. D'après la réduction de Jordan, nous avons que $\forall A \in \mathbf{M}_n(\mathbb{C})$, il existe $V, J \in \mathbf{M}_n(\mathbb{C})$ avec $V$ inversible et $J$ diagonale par blocs telle que :

$$A = VJV^{-1},$$

où

$$J = \begin{bmatrix} J_{m_1}(\lambda_1) & 0 & 0 & \cdots & 0 \\ 0 & J_{m_2}(\lambda_2) & 0 & \cdots & 0 \\ \vdots & \cdots & \ddots & \cdots & \vdots \\ 0 & \cdots & 0 & J_{m_{s-1}}(\lambda_{s-1}) & 0 \\ 0 & \cdots & \cdots & 0 & J_{m_s}(\lambda_s) \end{bmatrix},$$

avec

$$J_{m_i}(\lambda_i) = \begin{bmatrix} \lambda_i & 1 & 0 & \cdots & 0 \\ 0 & \lambda_i & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & \lambda_i & 1 \\ 0 & 0 & \cdots & 0 & \lambda_i \end{bmatrix} \in \mathbb{C}^{m_i \times m_i}, \quad 1 \leq i \leq s.$$

On a que

$$A^k = (VJV^{-1})^k = VJV^{-1}VJV^{-1}...VJV^{-1}$$

Comme $V^{-1}V = \mathbf{I}_n$ alors

$$A^k = VJ^kV^{-1}$$

et, puisque $J$ est diagonale par blocs,

$$J^k = \begin{bmatrix} J^k_{m_1}(\lambda_1) & 0 & 0 & \cdots & 0 \\ 0 & J^k_{m_2}(\lambda_2) & 0 & \cdots & 0 \\ \vdots & \cdots & \ddots & \cdots & \vdots \\ 0 & \cdots & 0 & J^k_{m_{s-1}}(\lambda_{s-1}) & 0 \\ 0 & \cdots & \cdots & 0 & J^k_{m_s}(\lambda_s) \end{bmatrix}.$$

Un résultat standard sur la puissance $k$ d'un bloc de Jordan $m_i \times m_i$ indique que, pour $k \geq m_i - 1$ :

$$J^k_{m_i}(\lambda_i) = \begin{bmatrix} \lambda_i^k & \binom{k}{1}\lambda_i^{k-1} & \binom{k}{2}\lambda_i^{k-2} & \cdots & \binom{k}{m_i-1}\lambda_i^{k-m_i+1} \\ 0 & \lambda_i^k & \binom{k}{1}\lambda_i^{k-1} & \cdots & \binom{k}{m_i-2}\lambda_i^{k-m_i+2} \\ \vdots & \vdots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & \lambda_i^k & \binom{k}{1}\lambda_i^{k-1} \\ 0 & 0 & \cdots & 0 & \lambda_i^k \end{bmatrix}.$$

Ainsi, si $\rho(A) < 1$, alors pour tout $i$, $|\lambda_i| < 1$. Ainsi, pour tout $i$, nous avons :

$$\lim_{k\to\infty} J^k_{m_i} = 0,$$

donc

$$\lim_{k\to\infty} J^k = 0.$$

et,

$$\lim_{k\to\infty} A^k = \lim_{k\to\infty} VJ^kV^{-1} = V\left(\lim_{k\to\infty} J^k\right)V^{-1} = 0.$$

$\square$

**Lemme 2.2 :** Pour toute norme de matrice $||\cdot||$, nous avons :

$$\rho(A) = \lim_{k\to\infty} \left\|A^k\right\|^{\frac{1}{k}}.$$

De plus, dans le cas d'une norme de matrice consistante, $\lim_{k\to\infty} \left\|A^k\right\|^{\frac{1}{k}}$ approche $\rho(A)$ par-dessus (en effet, dans ce cas, $\rho(A) \leq \left\|A^k\right\|^{\frac{1}{k}}$ pour tout k).

**Preuve :** Soit $\varepsilon > 0$, on pose :

$$A_+ = \tfrac{1}{\rho(A)+\varepsilon}A, \quad A_- = \tfrac{1}{\rho(A)-\varepsilon}A.$$

Ainsi,

$$\rho(A_+) = \tfrac{\rho(A)}{\rho(A)+\varepsilon}, \quad \rho(A_+) < 1 < \rho(A_-).$$

On applique le lemme 2.1 sur les limites des suites de puissances à $A_+$ :

$$\lim_{k\to\infty} A_+^k = 0.$$

Donc il existe un entier $N_+ \in \mathbb{N}$ tel que, pour tout $k \geq N_+$,

$$\left\|A_+^k\right\| < 1.$$

Alors,

$$\left\|A^k\right\|^{\frac{1}{k}} < \rho(A) + \varepsilon.$$

De même, le lemme 2.1 implique que $\left\|A_-^k\right\|$ n'est pas bornée et qu'il existe un entier $N_- \in \mathbb{N}$ tel que, pour tout $k \geq N_-$,

$$\left\|A_-^k\right\| > 1.$$

Alors,

$$\left\|A^k\right\|^{\frac{1}{k}} > \rho(A) - \varepsilon.$$

Soit $N = \max(N_+, N_-)$. Alors, $\forall \varepsilon > 0$, il existe un entier $N$ tel que, $\forall k \geq N$,

$$\rho(A) - \varepsilon < \left\|A^k\right\|^{\frac{1}{k}} < \rho(A) + \varepsilon,$$

$$\Leftrightarrow lim_{k\to\infty} \left\|A^k\right\|^{\frac{1}{k}} = \rho(A).$$

$\square$

**Lemme 2.3 :** Soient $\beta > 0$, $\alpha > 0$ et $A \in \mathbf{M}_n(\mathbb{R})$, une matrice symétrique, définie et positive. On pose $T = \begin{bmatrix} (1+\beta)\mathbf{I}_n - \alpha A & -\beta\mathbf{I}_n \\ \mathbf{I}_n & 0 \end{bmatrix}$ et $L = \lambda_1 \geq ...\lambda_n = l > 0$ où $\lambda_1, ..., \lambda_n$ sont les valeurs propres de $A$. Si $\beta > (1 - \sqrt{\alpha L})^2$ alors,

$$\rho(T) < \sqrt{\beta} \text{ où } \rho(T) \text{ est le rayon spectral de } T$$

**Preuve :** Soient $\beta > 0$, $\alpha > 0$ et $A \in \mathbf{M}_n(\mathbb{R})$, une matrice symétrique, définie et positive. On pose $T = \begin{bmatrix} (1+\beta)\mathbf{I}_n - \alpha A & -\beta\mathbf{I}_n \\ \mathbf{I}_n & 0 \end{bmatrix}$ et $L = \lambda_1 \geq ...\lambda_n = l > 0$ où $\lambda_1, ..., \lambda_n$ sont les valeurs propres de $A$. Supposons que $\beta > (1 - \sqrt{\alpha L})^2$, comme A est une matrice carrée symétrique définie positive, elle est diagonalisable et on peut la réécrire comme $A = PDP^T$ où P est orthogonale et D diagonale donc,

$$T = \begin{bmatrix} (1+\beta)\mathbf{I}_n - \alpha A & -\beta\mathbf{I}_n \\ \mathbf{I}_n & 0 \end{bmatrix}$$

$$= \begin{bmatrix} (1+\beta)\mathbf{I}_n - \alpha PDP^T & -\beta\mathbf{I}_n \\ \mathbf{I}_n & 0 \end{bmatrix}$$

Comme $T$ est diagonale et que D forme une base, on a

$$\|T\|_2 = \| \begin{bmatrix} (1+\beta)\mathbf{I}_n - \alpha PDP^T & -\beta\mathbf{I}_n \\ \mathbf{I}_n & 0 \end{bmatrix} \|_2$$

$$= \| \begin{bmatrix} (1+\beta)\mathbf{I}_n - \alpha D & -\beta\mathbf{I}_n \\ \mathbf{I}_n & 0 \end{bmatrix} \|_2$$

Comme T est une matrice de blocs qui sont diagonales, on peut réécrire :
exemple : si $n = 2$, $x_i = 1 + \beta - \alpha\lambda_i$ avec $i \in 1, .., n$

$$\|T\|_2 = \| \begin{bmatrix} x_1 & 0 & -\beta & 0 \\ 0 & x_2 & 0 & -\beta \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \|_2$$

Comme la norme est invariante par permutation, on peut réecrire :

$$L_3 \Leftrightarrow L_2$$

$$\|T\|_2 == \| \begin{bmatrix} x_1 & 0 & -\beta & 0 \\ 1 & 0 & 0 & 0 \\ 0 & x_2 & 0 & -\beta \\ 0 & 1 & 0 & 0 \end{bmatrix} \|_2$$

$$C_3 \Leftrightarrow C_2$$

$$= \| \begin{bmatrix} x_1 & -\beta & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & x_2 & -\beta \\ 0 & 0 & 1 & 0 \end{bmatrix} \|_2$$

On voit maintenant une diagonale de blocs,

$$\|T\|_2 = \| \begin{bmatrix} T_1 & 0 \\ 0 & T_2 \end{bmatrix} \|_2 \text{ , où } T_i = \begin{bmatrix} 1+\beta-\alpha\lambda_i & -\beta \\ 1 & 0 \end{bmatrix}$$

On permute comme précédemment jusqu'à qu'on a :

$$\|T\|_2 = \| \begin{bmatrix} T_1 & & 0 \\ & \ddots & \\ 0 & & T_n \end{bmatrix} \|_2 \text{ , où } T_i = \begin{bmatrix} 1+\beta-\alpha\lambda_i & -\beta \\ 1 & 0 \end{bmatrix} \text{ et } i \in 1, .., n$$

Calculons les valeurs propres de $T_i$,

$$\det \begin{bmatrix} 1+\beta-\alpha\lambda_i - X & -\beta \\ 1 & -X \end{bmatrix} = (1+\beta-\alpha\lambda_i - X)(-X) + \beta$$

$$= X^2 - X(1+\beta-\alpha\lambda_i) + \beta = 0,$$

$$\Delta = (1+\beta-\alpha\lambda_i)^2 - 4\beta$$

Si les racines sont complexes $\Leftrightarrow \Delta \leq 0$

$$\Leftrightarrow (1+\beta-\alpha\lambda_i)^2 \leq 4\beta$$

$$\Leftrightarrow 1+\beta-\alpha\lambda_i \leq 2\sqrt{\beta}$$

$$\Leftrightarrow 1-2\sqrt{\beta}+\beta \leq \alpha\lambda_i$$

$$\Leftrightarrow (1-\sqrt{\beta})^2 \leq \alpha\lambda_i$$

$$\Leftrightarrow 1-\sqrt{\beta} \leq \sqrt{\alpha\lambda_i}$$

$$\Leftrightarrow 1-\sqrt{\alpha\lambda_i} \leq \sqrt{\beta}$$

$$\Leftrightarrow (1-\sqrt{\alpha\lambda_i})^2 \leq \beta$$

$$\Leftrightarrow (1-\sqrt{\alpha L})^2 \leq \beta \text{ car } L = \max_{i\in 1,\dots,n} \lambda_i$$

Donc les racines du polynome caractéristique de $T_i$ sont forcements complexes. On a alors, $X_i = \frac{1}{2}(a_i \pm i b_i)$ avec $a_i = 1+\beta-\alpha\lambda_i$ et $b_i = \sqrt{-\Delta} = \sqrt{4\beta-(1+\beta-\alpha\lambda_i)^2}$. Ainsi,

$$|X_i| = \tfrac{1}{2}\sqrt{a_i^2+b_i^2}$$

$$= \tfrac{1}{2}\sqrt{(1+\beta-\alpha\lambda_i)^2 + (\sqrt{4\beta-(1+\beta-\alpha\lambda_i)^2})^2}$$

$$= \tfrac{1}{2}\sqrt{4\beta} = \sqrt{\beta}$$

$\sqrt{\beta}$ est donc plus grand que toutes les valeurs propres et on a,

$$\rho(T) < \sqrt{\beta} \text{ où } \rho(T) \text{ est le rayon spectral de } T$$

$\square$

**Théorème 2.1 :** Soit $f : \mathbb{R}^n \to \mathbb{R}$, une fonctionnelle quadratique de la forme $f(x) = \frac{1}{2}\langle Ax, x\rangle - \langle b, x\rangle + c$, où $A \in \mathbf{M}_n(\mathbb{R})$ est une matrice carrée symétrique, $b \in \mathbb{R}^n$ et $c \in \mathbb{R}$. Soit $\alpha > 0$, $\beta > 0$ tel que $\beta \geq (1-\sqrt{\alpha L})$, $L = \max_{i\in 1,\dots,n} \lambda_i$ où les $\lambda_i$ sont les valeurs propres de $A$. Alors,

$$||x_{k+1} - x^*|| \leq (\sqrt{\beta}+\epsilon_k)^k ||x_0 - x^*|| \text{ avec } \lim_{k\to\infty} \epsilon_k = 0$$

**Preuve :** On a, $x_{k+1} - x^* = x_k - \alpha \nabla f(x_k) + \beta(x_k - x_{k-1}) - x^*$
or $\nabla f(x_k) = Ax_k - b$ et $b = Ax^*$ donc $\nabla f(x_k) = Ax_k - Ax^*$ et

$$x_{k+1} - x^* = x_k - \alpha(Ax_k - Ax^*) + \beta(x_k - x_{k-1}) - x^*$$

$$= x_k - x^* - \alpha A(x_k - x^*) + \beta(x_k - x_{k-1})$$

$$= (\mathbf{I}_n - \alpha A)(x_k - x^*) + \beta(x_k - x_{k-1} + x^* - x^*)$$

$$= (\mathbf{I}_n - \alpha A)(x_k - x^*) - \beta(x_{k-1} - x^*) + \beta(x_k - x^*)$$

$$= ((1+\beta)\mathbf{I}_n - \alpha A)(x_k - x^*) - \beta(x_{k-1} - x^*)$$

On a,

$$\begin{bmatrix} x_{k+1} - x^* \\ x_k - x^* \end{bmatrix} = \begin{bmatrix} ((1+\beta)\mathbf{I}_n - \alpha A)(x_k - x^*) - \beta(x_{k-1} - x^*) \\ x_k - x^* \end{bmatrix}$$

$$= \begin{bmatrix} (1+\beta)\mathbf{I}_n - \alpha A & -\beta \\ \mathbf{I}_n & 0 \end{bmatrix} \begin{bmatrix} x_k - x^* \\ x_{k-1} - x^* \end{bmatrix}$$

On pose $y_k = \begin{bmatrix} x_{k+1} - x^* \\ x_k - x^* \end{bmatrix}$ et $T = \begin{bmatrix} (1+\beta)\mathbf{I}_n - \alpha A & -\beta \\ \mathbf{I}_n & 0 \end{bmatrix}$
donc $y_k = Ty_{k-1} = T^2 y_{k-2} = ... = T^k y_0$ et $\|y_k\|_2 = \|T^k y_0\|_2 \leq \|T^k\|_2 \|y_0\|_2$
D'après le lemme 2.2, $\|T^k\|_2 \leq (\rho(T) + \epsilon_k)^k$, supposons que $\beta \geq (1 - \sqrt{\alpha L})^2$ donc d'après le lemme 2.3, $\|T^k\|_2 \leq (\sqrt{\beta} + \epsilon_k)^k$.
Ainsi,

$$||x_{k+1} - x^*|| \leq (\sqrt{\beta} + \epsilon_k)^k ||x_0 - x^*|| \text{ avec } \lim_{k \to \infty} \epsilon_k = 0$$

□

Le théorème 2.2 montre que dans le cas des fonctionnelles quadratiques définies positives, l'algorithme de descente de gradient avec moment converge vers la solution optimale si on choisit les bonnes conditions sur $\alpha$ et $\beta$, c'est à dire que, $0 < \alpha < \frac{1}{L}$ et $(1 - \sqrt{\alpha L})^2 \leq \beta < 1$. La vitesse de convergence est exponentielle car

$$(\sqrt{\beta} + \epsilon_k)^k = e^{k \ln(\sqrt{\beta} + \epsilon_k)} = O(e^k)$$

Dans la prochaine section, nous étudierons la convergence de l'algorithme de descente de gradient avec moment pour une fonction convexe à gradient Lipschitz.

# 3 Convergence pour une fonction convexe à gradient Lipschitz

Dans cette partie, nous allons étudier la convergence de l'algorithme de descente de gradient avec moment (Heavy-ball algorithm) pour le cas des fonctions convexes à gradient Lipschitz.

On rappele que l'itération de Heavy-ball est

$$x_{k+1} = x_k - \alpha \nabla f(x_k) + \beta(x_k - x_{k-1}),$$

où $\alpha > 0$ est le pas de descente et $\beta > 0$ le moment.

**Définition 3.1 :** On dit que $f : \mathbb{R}^n \to \mathbb{R}$ est de classe $\mathcal{F}_L^{1,1}$, si $f$ est continue, diffèrentiable et à gradient continûment Lipschitz pour une constante $L$, c'est à dire que $\forall x, y \in \mathbb{R}^n$,

$$0 \leq f(y) - f(x) - \langle \nabla f(x), y - x \rangle \leq \tfrac{L}{2}\|x - y\|^2.$$

Avant de montrer la convergence, on doit voir trois lemmes qui seront utile pour montrer la convergence.

**Lemme 3.1 :** Toutes les conditions ci-dessous sont équivalentes à $\forall x, y \in \mathbb{R}^n$ , $\alpha \in [0, 1]$ , $f \in \mathcal{F}_L^{1,1}(\mathbb{R}^n)$ :

$$\text{(i) } 0 \leq f(y) - f(x) - \langle \nabla f(x), y - x \rangle \leq \tfrac{L}{2}\|x - y\|^2,$$

$$\text{(ii) } f(x) + \langle \nabla f(x), y - x \rangle + \tfrac{1}{2L}||\nabla f(x) - \nabla f(y)||^2 \leq f(y),$$

$$\text{(iii) } \tfrac{1}{L}||\nabla f(x) - \nabla f(y)||^2 \leq \langle \nabla f(x) - \nabla f(y), x - y \rangle,$$

$$\text{(iv) } \langle \nabla f(x) - \nabla f(y), x - y \rangle \leq L||x - y||^2$$

$$\text{(v) } \alpha f(x) + (1 - \alpha) f(y) \geq f(\alpha x + (1 - \alpha) y + \tfrac{\alpha(1-\alpha)}{2L}||\nabla f(x) - \nabla f(y)||^2$$

$$\text{(vi) } \alpha f(x) + (1 - \alpha) f(y) \leq f(\alpha x + (1 - \alpha) y) + \alpha(1 - \alpha)\tfrac{L}{2}||x - y||^2$$

**Preuve :**
(i) $\Rightarrow$ (ii)
On pose $\phi(y) = f(y) - \langle \nabla f(x), y \rangle$, on a que $\phi \in \mathcal{F}_L^{1,1}(\mathbb{R}^n)$ et $y^* = x$ car $\phi(x) = 0$ donc,

$$0 \leq \phi(y) - \phi(x) - \langle \nabla \phi(x), y - x \rangle \leq \tfrac{L}{2}\|x - y\|^2$$

$$\Rightarrow \phi(y) \leq \tfrac{L}{2}||x - y||^2 + \phi(x) + \langle \nabla \phi(x), y - x \rangle$$

On pose $x = y$ et $y = y - \frac{1}{L}\nabla \phi(y)$ donc,

$$\phi(y - \tfrac{1}{L}) \leq \tfrac{L}{2}||y - y - \tfrac{1}{L}||^2 + \phi(y) + \langle \nabla\phi(y), y - \tfrac{1}{L} - y\rangle$$

$$\leq \tfrac{1}{2L}||\nabla\phi(y)||^2 + \phi(y) - \tfrac{1}{L}||\nabla\phi(y)||^2$$

$$\leq \phi(y) - \tfrac{1}{2L}||\nabla\phi(y)||^2$$

Comme $y^* = x$,

$$\phi(x) \leq \phi(y) - \tfrac{1}{2L}||\nabla\phi(y)||^2$$

On remplace $\phi$,

$$f(x) - \langle \nabla f(x), x\rangle \leq f(y) - \langle \nabla f(x), y\rangle - \tfrac{1}{2L}||\nabla f(y) - \nabla f(x)||^2$$

$$\Leftrightarrow f(x) + \langle \nabla f(x), y - x\rangle + \tfrac{1}{2L}||\nabla f(x) - \nabla f(y)||^2 \leq f(y)$$

(ii) $\Rightarrow$ (iii)
On a que,

$$f(x) + \langle \nabla f(x), y - x\rangle + \tfrac{1}{2L}||\nabla f(x) - \nabla f(y)||^2 \leq f(y)$$

On inverse x et y,

$$f(y) + \langle \nabla f(y), x - y\rangle + \tfrac{1}{2L}||\nabla f(y) - \nabla f(x)||^2 \leq f(x)$$

On somme les deux inégalitées,

$$f(y) + \langle \nabla f(y), x - y\rangle + \tfrac{1}{2L}||\nabla f(y) - \nabla f(x)||^2 + f(x) + \langle \nabla f(x), y - x\rangle + \tfrac{1}{2L}||\nabla f(x) - \nabla f(y)||^2 \leq f(x) + f(y)$$

$$\Leftrightarrow \tfrac{1}{L}||\nabla f(x) - \nabla f(y)||^2 + \langle \nabla f(y) - \nabla f(x), x - y \leq 0$$

$$\Leftrightarrow \tfrac{1}{L}||\nabla f(x) - \nabla f(y)||^2 \leq \langle \nabla f(x) - \nabla f(y), x - y\rangle$$

(iii) $\Rightarrow$ (iv)
On a que,

$$\tfrac{1}{L}||\nabla f(x) - \nabla f(y)||^2 \leq \langle \nabla f(x) - \nabla f(y), x - y\rangle$$

$$\leq ||\nabla f(x) - \nabla f(y)|| * ||x - y|| \text{ d'après l'inégalitée de Cauchy-Schwartz}$$

donc,

$$||\nabla f(x) - \nabla f(y)|| \leq L||x - y||$$

$$\Leftrightarrow ||x - y|| * ||\nabla f(x) - \nabla f(y)|| \leq L||x - y||^2$$

Ainsi,

$$\langle \nabla f(x) - \nabla f(y), x - y\rangle \leq L||x - y||^2$$

(iv) $\Rightarrow$ (i)
On a que,

$$f(y) - f(x) - \langle \nabla f(x), y - x \rangle = \int_0^1 \langle \nabla f(x + r(y-x)) - \nabla f(x), y - x \rangle dr$$

$$\leq \frac{L}{2}||y - x||^2 \text{ d'après (iv)}$$

(ii) $\Rightarrow$ (v)
On pose $x_\alpha = \alpha x + (1-\alpha)y$ donc,

$$f(x_\alpha) + \langle \nabla f(x_\alpha), x - x_\alpha \rangle + \frac{1}{2L}||\nabla f(x) - \nabla f(x_\alpha)||^2 \leq f(x)$$

et

$$f(x_\alpha) + \langle \nabla f(x_\alpha), y - x_\alpha \rangle + \frac{1}{2L}||\nabla f(x) - \nabla f(x_\alpha)||^2 \leq f(y)$$

On ajoute les deux inégalitées multipliées par $\alpha$ et $1-\alpha$ respectivement,

$$f(x_\alpha) + \alpha(1-\alpha)\langle \nabla f(x_\alpha), x - y \rangle + \frac{\alpha}{2L}||\nabla f(x) - \nabla f(x_\alpha)||^2 + \alpha(1-\alpha)\langle \nabla f(x_\alpha), y - x \rangle + \frac{1-\alpha}{2L}||\nabla f(y) - \nabla f(x_\alpha)||^2 \leq \alpha f(x) + (1-\alpha)f(y)$$

$$\Leftrightarrow f(x_\alpha) + \frac{\alpha}{2L}||\nabla f(x) - \nabla f(x_\alpha)||^2 + \frac{1-\alpha}{2L}||\nabla f(y) - \nabla f(x_\alpha)||^2 \leq \alpha f(x) + (1-\alpha)f(y)$$

On utilise l'inégalité,

$$\alpha(1-\alpha)||g_1 - g_2||^2 \leq \alpha||g_1 - u||^2 + (1-\alpha)||g_2 - u||^2$$

donc,

$$f(x_\alpha) + \frac{\alpha(1-\alpha)}{2L}||\nabla f(x) - \nabla f(y)||^2 \leq \alpha f(x) + (1-\alpha)f(y)$$

$$\text{ie } f(\alpha x + (1-\alpha)y) + \frac{\alpha(1-\alpha)}{2L}||\nabla f(x) - \nabla f(y)||^2 \leq \alpha f(x) + (1-\alpha)f(y)$$

(v) $\Rightarrow$ (ii)
On pose $g(\alpha) = f(\alpha x + (1-\alpha)y)$. Comme $f$ est différentiable, $g$ est dérivable avec $g'(\alpha) = df(\alpha x + (1-\alpha)y)(x - y) = \langle \nabla f(\alpha x + (1-\alpha)y), x - y \rangle$ donc,

$$g'(1) = \langle \nabla f(x), x - y \rangle$$

or,

$$g'(1) = \lim_{\alpha \to 1} \frac{g(1) - g(\alpha)}{1 - \alpha}$$

et

$$g(1) - g(\alpha) = f(x) - f(\alpha x + (1-\alpha)y)$$

d'où,

$$\langle \nabla f(x), x - y \rangle = \lim_{\alpha \to 1} \frac{f(x) - f(\alpha x + (1-\alpha)y)}{1 - \alpha}$$

On a que,

$$\alpha f(x) + (1-\alpha)f(y) \geq f(\alpha x + (1-\alpha)y) + \tfrac{\alpha(1-\alpha)}{2L}||\nabla f(x) - \nabla f(y)||^2$$

$$\Leftrightarrow f(y) - f(x) + \tfrac{f(x)-f(\alpha x+(1-\alpha)y)}{1-\alpha} \geq \tfrac{\alpha}{2L}||\nabla f(x) - \nabla f(y)||^2$$

On fait tendre $\alpha$ vers 1 et

$$f(y) - f(x) + \langle \nabla f(x), x-y\rangle \geq \tfrac{1}{2L}||\nabla f(x) - \nabla f(y)||^2$$

$$\Leftrightarrow f(y) \geq f(x) + \langle \nabla f(x), y-x\rangle + \tfrac{1}{2L}||\nabla f(x) - \nabla f(y)||^2$$

(i) $\Rightarrow$ (vi)
On pose $x_\alpha = \alpha x + (1-\alpha)y$ donc,

$$f(x) \leq f(x_\alpha) + \langle \nabla f(x_\alpha), x - x_\alpha\rangle + \tfrac{L}{2}||x - x_\alpha||^2$$

et

$$f(y) \leq f(x_\alpha) + \langle \nabla f(x_\alpha), y - x_\alpha\rangle + \tfrac{L}{2}||y - x_\alpha||^2$$

On ajoute les deux inégalitées multipliées par $\alpha$ et $1\alpha$ respectivement,

$$\alpha f(x) + (1-\alpha)f(y) \leq$$
$$f(x_\alpha)+\alpha(1-\alpha)\langle \nabla f(x_\alpha), x-y\rangle+\alpha\tfrac{L}{2}||(1-\alpha)(x-y)||^2+\alpha(1-\alpha)\langle \nabla f(x_\alpha), y-x\rangle+(1-\alpha)\tfrac{L}{2}||\alpha(y-x)||^2$$

$$\Leftrightarrow \alpha f(x) + (1-\alpha)f(y) \leq f(x_\alpha) + \alpha(1-\alpha)^2\tfrac{L}{2}||x-y||^2 + (1-\alpha)\alpha^2\tfrac{L}{2}||y-x||^2$$

$$\Leftrightarrow \alpha f(x) + (1-\alpha)f(y) \leq f(\alpha x + (1-\alpha)y) + \alpha(1-\alpha)\tfrac{L}{2}||x-y||^2$$

(vi) $\Rightarrow$ (i)
On pose $g(\alpha) = f(\alpha x + (1-\alpha)y)$. Comme $f$ est différentiable, $g$ est dérivable avec $g'(\alpha) = df(\alpha x + (1-\alpha)y)(x-y) = \langle \nabla f(\alpha x + (1-\alpha)y), x-y\rangle$ donc,

$$g'(1) = \langle \nabla f(x), x-y\rangle$$

or,

$$g'(1) = \lim_{\alpha\to 1} \frac{g(1) - g(\alpha)}{1-\alpha}$$

et

$$g(1) - g(\alpha) = f(x) - f(\alpha x + (1-\alpha)y)$$

d'où,

$$\langle \nabla f(x), x-y\rangle = \lim_{\alpha\to 1} \frac{f(x) - f(\alpha x + (1-\alpha)y)}{1-\alpha}$$

On a que,

$$\alpha f(x) + (1-\alpha)f(y) \leq f(\alpha x + (1-\alpha)y) + \alpha(1-\alpha)\tfrac{L}{2}||x-y||^2$$

$$\Leftrightarrow f(y) - f(x) + \tfrac{f(x)-f(\alpha x+(1-\alpha)y)}{1-\alpha} \leq \tfrac{\alpha L}{2}||x-y||^2$$

On fait tendre $\alpha$ vers 1 et

$$f(y) - f(x) + \langle \nabla f(x), x-y \rangle \leq \tfrac{L}{2}||x-y||^2$$

$$\Leftrightarrow f(y) \leq f(x) + \langle \nabla f(x), y-x \rangle + \tfrac{L}{2}||x-y||^2$$

$\square$

**Lemme 3.2 :** Si pour tout $N \geq 2$ et pour tout $(\theta_1, ..., \theta_N)$ tel que $\sum_{k=1}^{N} \theta_k = 1$ , $\forall k \in 1, ..., N$ , $\theta_k \geq 0$ et $f$ convexe, alors on a pour tout $x_1, ..., x_N \in \mathbb{R}$

$$f(\sum_{k=1}^{N} \theta_k x_k) \leq \sum_{k=1}^{N} \theta_k f(x_k)$$

**Preuve :** On pose pour $N \geq 2$, $P(N)$ :"Si $\forall \theta_k$ , $\forall x_k$ , on a $\sum_{k=1}^{N} \theta_k = 1$ et $\theta_k \geq 0$ alors, $f(\sum_{k=1}^{N} \theta_k x_k) \leq \sum_{k=1}^{N} \theta_k f(x_k)$"
Initialisation : Pour N=2, soit $\theta_1, \theta_2 \geq 0$ tel que $\theta_1 + \theta_2 = 1$. On a, $\theta_2 = 1 - \theta_1$.
Soit, $x_1, x_2 \in \mathbb{R}$, comme f est convexe alors,

$$f(\theta_1 x_1 + (1-\theta_1)x_2) \leq \theta_1 f(x_1) + (1-\theta_1)f(x_2)$$

donc P(2) est vraie et la propriété est initialisée.
Hérédité : Supposons P(N), montrons P(N+1)
Comme on a P(N),

$$f(\sum_{k=1}^{N} \theta_k x_k) \leq \sum_{k=1}^{N} \theta_k f(x_k).$$

Soit $(t_1, ..., t_{N+1})$ tel que $\sum_{k=1}^{N} t_k = 1$ et $t_k \geq 0$.
Si $t_{N+1} = 1$,

$$f(\sum_{k=1}^{N+1} t_k x_k) = f(0 * x_N + t_{N+1} x_{N+1})$$

$$\leq 0 * f(x_N) + t_{N+1} f(x_{N+1}) \text{ d'après P(2)}$$

$$\leq \sum_{k=1}^{N+1} t_k f(x_k)$$

Si $t_{N+1} \neq 1$, on pose $t'_i = \frac{t_i}{1-t_{N+1}}$ pour $i \in 1, ..., N$ donc $\sum_{k=1}^{N} t'_k = 1$.

$$f(\sum_{k=1}^{N+1} t_k x_k) = f(\sum_{k=1}^{N} t_k x_k + t_{N+1} x_{N+1})$$

$$= f(\sum_{k=1}^{N} (1 - t_{N+1}) t'_k x_k + t_{N+1} x_{N+1})$$

$$= f((1 - t_{N+1}) \sum_{k=1}^{N} t'_k x_k + t_{N+1} x_{N+1})$$

$$\leq (1 - t_{N+1}) f(\sum_{k=1}^{N} t'_k x_k) + t_{N+1} f(x_{N+1}) \text{ d'après } P(2)$$

$$\leq (1 - t_{N+1})(\sum_{k=1}^{N} t'_k f(x_k) + t_{N+1} f(x_{N+1}) \text{ d'apès } P(N)$$

$$\leq \sum_{k=1}^{N+1} t_k f(x_k)$$

Ainsi P(N+1) donc la propriétée est héréditaire et on a, si pour tout $N \geq 2$ et pour tout $(\theta_1, ..., \theta_N)$ tel que $\sum_{k=1}^{N} \theta_k = 1$ , $\forall k \in 1, ..., N$ , $\theta_k \geq 0$ et $f$ convexe, alors on a pour tout $x_1, ..., x_N \in \mathbb{R}$

$$f(\sum_{k=1}^{N} \theta_k x_k) \leq \sum_{k=1}^{N} \theta_k f(x_k)$$

$\square$

**Lemme 3.3 :** Si $f \in \mathcal{F}_L^{1,1}$ alors,

$$f(x_0) - f^* \leq \frac{L}{2} ||x_0 - x^*||^2$$

**Preuve :** Supposons que $f \in \mathcal{F}_L^{1,1}$. On a alors,

$$0 \leq f(y) - f(x) - \langle \nabla f(x), y - x \rangle \leq \tfrac{L}{2}\|x - y\|^2$$

On pose $y = x_0$ et $x = x^*$ donc,

$$0 \leq f(x_0) - f^* - \langle \nabla f(x^*), x_0 - x^* \rangle \leq \tfrac{L}{2}\|x^* - x_0\|^2 \text{ car } \nabla f(x^*) = 0$$

d'où

$$f(x_0) - f^* \leq \tfrac{L}{2}\|x^* - x_0\|^2 = \|x_0 - x^*\|^2$$

$\square$

**Théorème 3.1 :** Si $f \in \mathcal{F}_L^{1,1}$ et que $\beta \in [0, 1[$ , $\alpha \in ]0, \frac{2(1-\beta)}{L}[$ alors,

$$f(\bar{x}_t) - f^* \leq \begin{cases} \frac{||x_0 - x^*||^2}{2(T+1)}(\frac{L\beta}{1-\beta} + \frac{1-\beta}{\alpha}) & \text{si } \alpha \in ]0, \frac{1-\beta}{L}[ \\ \frac{||x_0 - x^*||^2}{2(T+1)(2(1-\beta)-\alpha L)}(L\beta + \frac{(1-\beta)^2}{\alpha}) & \text{si } \alpha \in ]\frac{1-\beta}{L}, \frac{2(1-\beta)}{L}[ \end{cases}$$

où $\bar{x}_t = \frac{1}{T+1}\sum_{k=0}^{N}$ est la moyenne de Cesáro

**Preuve :** Supposons que $\beta \in [0, 1[$ et $k \in \mathbb{N}$. On pose $p_k = \frac{\beta}{1-\beta}(x_k - x_{k-1})$ alors,

$$x_{k+1} + p_{k+1} = x_{k+1} + \tfrac{\beta}{1-\beta}(x_k - x_{k-1})$$

$$= \tfrac{1}{1-\beta}x_{k+1} + \tfrac{\beta}{1-\beta}x_k$$

$$= \tfrac{1}{1-\beta}(x_k - \alpha\nabla f(x_k) + \beta(x_k - x_{k-1})) + \tfrac{\beta}{1-\beta}x_k$$

$$= \tfrac{1}{1-\beta}x_k - \tfrac{\alpha}{1-\beta}\nabla f(x_k) - \tfrac{\beta}{1-\beta}x_{k-1}$$

$$= \tfrac{1-\beta+\beta}{1-\beta}x_k - \tfrac{\alpha}{1-\beta}\nabla f(x_k) - \tfrac{\beta}{1-\beta}x_{k-1}$$

$$= \tfrac{1-\beta+\beta}{1-\beta}x_k - \tfrac{\alpha}{1-\beta}\nabla f(x_k) - \tfrac{\beta}{1-\beta}x_{k-1}$$

$$= x_k + \tfrac{\beta}{1-\beta}(x_k - x_{k-1}) - \tfrac{\alpha}{1-\beta}\nabla f(x_k)$$

$$= x_k + p_k - \tfrac{\alpha}{1-\beta}\nabla f(x_k)$$

Donc,

$$||x_{k+1} + p_{k+1} - x^*||^2 = ||x_k + p_k - \tfrac{\alpha}{1-\beta}\nabla f(x_k) - x^*||^2$$

$$= ||x_k + p_k - x^*||^2 - 2\langle x_k + p_k - x^*, \tfrac{\alpha}{1-\beta}\nabla f(x_k)\rangle + ||\tfrac{\alpha}{1-\beta}\nabla f(x_k)||^2$$

$$= ||x_k + p_k - x^*||^2 - \frac{2\alpha}{1-\beta}\langle x_k + p_k - x^*, \nabla f(x_k)\rangle + (\frac{\alpha}{1-\beta})^2||\nabla f(x_k)||^2$$

$$= ||x_k + p_k - x^*||^2 - \frac{2\alpha}{1-\beta}\langle x_k - x^*, \nabla f(x_k)\rangle + (\frac{\alpha}{1-\beta})^2||\nabla f(x_k)||^2 - \frac{2\alpha}{1-\beta}\langle p_k, \nabla f(x_k)\rangle$$

$$= ||x_k + p_k - x^*||^2 - \frac{2\alpha}{1-\beta}\langle x_k - x^*, \nabla f(x_k)\rangle + (\frac{\alpha}{1-\beta})^2||\nabla f(x_k)||^2 - \frac{2\alpha\beta}{(1-\beta)^2}\langle x_k - x_{k-1}, \nabla f(x_k)\rangle$$

Comme $f \in \mathcal{F}_L^{1,1}$, en utilisant le lemme 3.1, en posant $x = x_k$ et $y = x^*$ on a,

$$\text{(i) } \frac{1}{L}||\nabla f(x_k)||^2 \leq \langle x_k - x^*, \nabla f(x_k)\rangle,$$

$$\text{(ii) } f(x_k) - f^* + \frac{1}{2L}||\nabla f(x_k)||^2 \leq \langle x_k - x^*, \nabla f(x_k)\rangle,$$

$$\text{(iii) } f(x_k) - f(x_{k-1} \leq \langle x_k - x_{k-1}, \nabla f(x_k)\rangle$$

Donc,

$$||x_{k+1} + p_{k+1} - x^*|| \leq ||x_k + p_k - x^*|| - \frac{2\alpha}{1-\beta}(1-\lambda)\langle x_k - x^*, \nabla f(x_k)\rangle - \frac{2\alpha}{1-\beta}\lambda\langle x_k - x^*, \nabla f(x_k)\rangle + (\frac{\alpha}{1-\beta})^2||\nabla f(x_k)||^2 - \frac{2\alpha\beta}{(1-\beta)^2}\langle x_k - x_{k-1}, \nabla f(x_k)\rangle$$

où $\lambda \in ]0, 1]$ est un paramètre que nous utiliserons pour équilibrer le poids entre les inégalitées (i) et (ii).

$$||x_{k+1} + p_{k+1} - x^*|| \leq ||x_k + p_k - x^*|| - \frac{(2\alpha)(1-\lambda)}{(1-\beta)L}||\nabla f(x_k)||^2 - \frac{2\alpha\lambda}{1-\beta}(f(x_k) - f^* + \frac{1}{2L}||\nabla f(x_k)||^2) + (\frac{\alpha}{1-\beta})^2||\nabla f(x_k)||^2 - \frac{2\alpha\beta}{(1-\beta)^2}(f(x_k) - f(x_{k-1})$$

Ansi, on a

$$||x_{k+1} + p_{k+1} - x^*|| \leq ||x_k + p_k - x^*|| - \frac{(2\alpha)(1-\lambda)}{(1-\beta)L}||\nabla f(x_k)||^2 - \frac{2\alpha\lambda}{1-\beta}(f(x_k) - f^*) - \frac{\alpha\lambda}{L(1-\beta)}||\nabla f(x_k)||^2 - \frac{2\alpha\beta}{(1-\beta)^2}(f(x_k) - f(x_{k-1}) + (\frac{\alpha}{1-\beta})^2||\nabla f(x_k)||^2$$

De plus,

$$-\frac{2\alpha\lambda}{1-\beta}(f(x_k) - f^*) - \frac{2\alpha\beta}{(1-\beta)^2}(f(x_k) - f(x_{k-1}))$$

$$= -(\frac{2\alpha\lambda}{1-\beta})(\lambda + \frac{\beta}{1-\beta})f(x_k) + (\frac{2\alpha}{1-\beta})(\lambda + \frac{\beta}{1-\beta})(\frac{2\alpha\beta}{(1-\beta)^2})f^* + \frac{2\alpha\beta}{(1-\beta)^2}f(x_{k-1})$$

$$= (\frac{2\alpha\lambda}{1-\beta})(\lambda + \frac{\beta}{1-\beta})(f^* - f(x_k)) + \frac{2\alpha\beta}{(1-\beta)^2}(f(x_{k-1}) - f^*)$$

et

$$-\frac{2\alpha(1-\lambda)}{L(1-\beta)}||\nabla f(x_k)||^2 - \frac{\alpha\lambda}{L(1-\beta)}||\nabla f(x_k)||^2 + (\frac{\alpha}{1-\beta})^2||\nabla f(x_k)||^2$$

$$= (\frac{\alpha(-2+\lambda)}{L(1-\beta)} + (\frac{\alpha}{1-\beta})^2)||\nabla f(x_k)||^2$$

$$= \frac{\alpha}{1-\beta}(\frac{\alpha}{1-\beta} - \frac{2-\lambda}{L})||\nabla f(x_k)||^2$$

Donc,

$$\tfrac{2\alpha}{1-\beta}(\lambda+\tfrac{\beta}{1-\beta})(f(x_k)-f^*)+||x_{k+1}+p_{k+1}-x^*||^2 \leq$$
$$\tfrac{2\alpha\beta}{(1-\beta)^2}(f(x_{k-1})-f^*)+||x_k+p_k-x^*||^2+(\tfrac{\alpha}{1-\beta})(\tfrac{\alpha}{1-\beta}-\tfrac{2-\lambda}{L})||\nabla f(x_k)||^2$$

Si $\alpha \in [0, \frac{(2-\lambda)(1-\beta)}{L}]$, le dernier terme est négatif donc on peut le retirer et on somme pour $k = 1, ..., T$, on a

$$\tfrac{2\alpha\lambda}{(1-\beta)}\sum_{k=0}^{T}(f(x_k)-f^*)+\sum_{k=0}^{T}(\frac{2\alpha\beta}{(1-\beta)^2}(f(x_k)-f^*)+||x_{k+1}+p_{k+1}-x^*||^2) \leq$$
$$\sum_{k=0}^{T}(\frac{2\alpha\beta}{(1-\beta)^2}(f(x_{k-1})-f^*)+||x_k+p_k-x^*||^2)$$

donc,

$$\tfrac{2\alpha\lambda}{(1-\beta)}\sum_{k=0}^{T}(f(x_k)-f^*)+\frac{2\alpha\beta}{(1-\beta)^2}(f(x_T)-f^*)+||x_{T+1}+p_{T+1}-x^*||^2) \leq$$
$$\frac{2\alpha\beta}{(1-\beta)^2}(f(x_{-1})-f^*)+||x_0+p_0-x^*||^2$$

ainsi,

$$\tfrac{2\alpha\lambda}{(1-\beta)}\sum_{k=0}^{T}(f(x_k)-f^*) \leq \frac{2\alpha\beta}{(1-\beta)^2}(f(x_0)-f^*)+||x_0-x^*||^2$$

car $x_{-1} = x_0$, $p_0 = \frac{\beta}{1-\beta}(x_0 - x_0) = 0$ et $||x_{T+1}+p_{T+1}-x^*||^2 \geq 0$.

Comme $f$ est convexe, $\displaystyle\sum_{k=0}^{T}\frac{1}{T+1} = 1$ et $\frac{1}{T+1} \geq 0$ alors, d'après le lemme 3.2

$$(T+1)f(\bar{x}_T) = (T+1)f(\tfrac{1}{T+1}\sum_{k=0}^{T}x_k)$$

$$\leq \tfrac{T+1}{T+1}\sum_{k=0}^{T}f(x_k) = \sum_{k=0}^{T}f(x_k)$$

Donc,

$$\tfrac{2\alpha\lambda}{(1-\beta)}\sum_{k=0}^{T}(f(x_k)-f^*) \leq \frac{2\alpha\beta}{(1-\beta)^2}(f(x_0)-f^*)+||x_0-x^*||^2$$

$$\Leftrightarrow \sum_{k=0}^{T}(f(x_k)-f^*) \leq \frac{\beta}{\lambda(1-\beta)}(f(x_0)-f^*)+\frac{(1-\beta)}{2\alpha\lambda}||x_0-x^*||^2$$

$$\Rightarrow (T+1)(f(\bar{x}_T)-f^*) \leq \tfrac{\beta}{\lambda(1-\beta)}(f(x_0)-f^*)+\tfrac{(1-\beta)}{2\alpha\lambda}||x_0-x^*||^2$$

$$\Rightarrow f(\bar{x}_T) - f^* \leq \frac{1}{T+1}(\frac{\beta}{\lambda(1-\beta)}(f(x_0) - f^*) + \frac{(1-\beta)}{2\alpha\lambda}||x_0 - x^*||^2)$$

De plus, d'après le lemme 3.3,

$$f(x_0) - f^* \leq \frac{L}{2}||x_0 - x^*||^2$$

donc,

$$\Leftrightarrow f(\bar{x}_T) - f^* \leq \frac{1}{T+1}(\frac{\beta}{\lambda(1-\beta)}\frac{L}{2}||x_0 - x^*||^2 + \frac{(1-\beta)}{2\alpha\lambda}||x_0 - x^*||^2)$$

$$\leq \frac{1}{T+1}||x_0 - x^*||^2(\frac{\beta L}{2\lambda(1-\beta)} + \frac{1-\beta}{2\alpha\lambda})$$

Si $\alpha \in ]0, \frac{1-\beta}{L}]$, on pose $\lambda = 1$ et on a,

$$f(\bar{x}_T) - f^* \leq \frac{||x_0 - x^*||^2}{2(T+1)}(\frac{\beta L}{1-\beta} + \frac{1-\beta}{2\alpha})$$

Si $\alpha \in [\frac{1-\beta}{L}, \frac{2(1-\beta)}{L}[$, on pose $\lambda = 2 - \frac{\alpha L}{1-\beta}$ et on a,

$$f(\bar{x}_T) - f^* \leq \frac{||x_0 - x^*||^2}{2(T+1)}(\frac{\beta L}{(2-\frac{\alpha L}{1-\beta})(1-\beta)} + \frac{1-\beta}{\alpha(2-\frac{\alpha L}{1-\beta})})$$

$$\leq \frac{||x_0 - x^*||^2}{2(T+1)(2(1-\beta)-\alpha L)}(\beta L + \frac{(1-\beta)^2}{\alpha})$$

□

Le théorème 3.1 montre que dans le cas des fonctionS convexe à gradient LipschitZ, l'algorithme de descente de gradient avec moment converge vers la solution optimale si on choisit les bonnes conditions sur $\alpha$ et $\beta$, c'est à dire que, $0 \leq \beta < 1$ et $0 < \alpha < \frac{2(1-\beta)}{L}$. La vitesse de convergence est linéaire car $O(k)$

Dans la prochaine section, nous étudierons la convergence de l'algorithme de descente de gradient avec moment pour une fonction fortement convexe à gradient Lipschitz.

# 4 Convergence pour une fonction fortement convexe

Dans cette partie, nous allons étudier la convergence de l'algorithme de descente de gradient avec moment (Heavy-ball algorithm) pour une fonction fortement convexe.

On rappele que l'itération de Heavy-ball est

$$x_{k+1} = x_k - \alpha \nabla f(x_k) + \beta(x_k - x_{k-1}),$$

où $\alpha > 0$ est le pas de descente et $\beta > 0$ le moment.

**Définition :** Soit $f : \mathbb{R}^n \to \mathbb{R}$, on dit que $f$ appartient à $S_{1,1}^{\mu,L}$, si la fonction $f$ appartient à la classe $F_{1,1}^L$ et si pour tous les $x, y \in \mathbb{R}^n$, $f$ est également fortement convexe avec un module $\mu > 0$, c'est-à-dire :

$$\tfrac{\mu}{2}\|x - y\|_2^2 \leq f(y) - f(x) - \langle \nabla f(x), y - x \rangle, \quad \forall x, y \in \mathbb{R}^n$$

**Théorème 4.1 :** Supposons que $f$ appartient à $S_{1,1}^{\mu,L}$ et que

$$\alpha \in ]0, \tfrac{2}{L}[,\ 0 \leq \beta \leq \tfrac{1}{2}(\tfrac{\mu\alpha}{2} + \sqrt{\tfrac{\mu^2\alpha^2}{4} + 4(1 - \tfrac{\alpha L}{2})})[.$$

Alors,

$$f(x_k) - f^* \leq q^k h(f(x_0) - f^*), \text{ où } q \in [0, 1[ \text{ et } h \in \mathbb{R}.$$

Le début de la preuve va beaucoup ressembler à la preuve du théorème 3.1. Mais avant de passer à la preuve de ce théorème, on doit montrer ces deux lemmes qui seront utlises à la preuve du théorème.

**Lemme 4.1 :** Soient $\{A_k\}_{k\geq 0}$ et $\{B_k\}_{k\geq 0}$ des suites de nombres réels positifs vérifiant

$$A_{k+1} + bB_{k+1} \leq a_1 A_k + a_2 A_{k-1} + cB_k, \quad k \in \mathbb{N}_0$$

avec des constantes $a_1$, $a_2$, $b \in \mathbb{R}^+$ et $c \in \mathbb{R}$. De plus, supposons que $A_{-1} = A_0$, $a_1 + a_2 < 1$ et $c < b$. Alors, la séquence $\{A_k\}_{k\geq 0}$ satisfait

$$A_k \leq q^k \left((q - a_1 + 1)A_0 + bB_0\right)$$

où $q \in [0, 1[$ est défini par $q = \max\left(\frac{c}{b}, \frac{a_1 + \sqrt{a_1^2 + 4a_2}}{2}\right)$

**Preuve :** Soient $\gamma \geq 0$ et $k \in \mathbb{N}$. Soient $\{A_k\}_{k\geq 0}$ et $\{B_k\}_{k\geq 0}$ des suites de nombres réels positifs vérifiant

$$A_{k+1} + bB_{k+1} \leq a_1 A_k + a_2 A_{k-1} + cB_k, \quad k \in \mathbb{N}_0$$

avec des constantes $a_1$, $a_2$, $b \in \mathbb{R}^+$ et $c \in \mathbb{R}$. De plus, supposons que $A_{-1} = A_0$, $a_1 + a_2 < 1$ et $c < b$. On a,

$$A_{k+1} + bB_{k+1} + \gamma A_k \leq a_1 A_k + a_2 A_{k-1} + cB_k, \quad k \in \mathbb{N}_0 + \gamma A_k$$

$$\leq (a_1 + \gamma)(A_k + \tfrac{a_2}{a_1+\gamma} A_{k-1} + \tfrac{c}{a_1+\gamma} B_k)$$

Si $\frac{a_2}{a_1+\gamma} \leq \gamma$ et $\frac{c}{a_1+\gamma}$ alors,

$$A_{k+1} + bB_{k+1} + \gamma A_k \leq (a_1 + \gamma)(A_k + \gamma A_{k-1} + bB_k)$$

Or, $\frac{a_2}{a_1+\gamma} \leq \gamma \Leftrightarrow 0 \leq \gamma^2 + \gamma a_1 - a_2$

$$\Delta = a_1^2 + 4a_2$$

Donc les solutions de $\gamma^2 + \gamma a_1 - a_2 = 0$ sont

$$\gamma_1 = \tfrac{-a_1+\sqrt{a_1^2+4a_2}}{2} > 0 \text{ et } \gamma_2 = \tfrac{-a_1-\sqrt{a_1^2+4a_2}}{2} < 0$$

$$\Leftrightarrow \gamma \geq \gamma_1 = \tfrac{-a_1+\sqrt{a_1^2+4a_2}}{2}$$

et $\frac{c}{a_1+\gamma} \leq b \Leftrightarrow \frac{c}{b} - a_1$
On pose alors, $\gamma = \max\left(\frac{-a_1+\sqrt{a_1^2+4a_2}}{2}, \frac{c}{b} - a_1, 0\right)$, on a donc bien $\frac{a_2}{a_1+\gamma} \leq \gamma$ et $\frac{c}{a_1+\gamma} \leq b$
On pose $q = a_1 + \gamma = \max\left(\frac{a_1+\sqrt{a_1^2+4a_2}}{2}, \frac{c}{b}\right)$
Comme $a_1 + a_2 < 1 \Leftrightarrow a_2 < 1 - a_1 \Leftrightarrow a_1^2 + 4a_2 < 4 - 4a_1 + a_1^2 \Leftrightarrow a_1^2 + 4a_2 < (2 - a_1)^2 \Leftrightarrow \sqrt{a_1^2 + 4a_2} < 2 - a_1 \Leftrightarrow 0 \leq \frac{a_1+\sqrt{a_1^2+4a_2}}{2} < 1$
et $\frac{c}{b} \in ]-\infty, 1[$ car $c \in \mathbb{R}$, $b \in \mathbb{R}^+$ et$c < n$ alors, $q < 1$
Ansi,

$$A_{k+1} + \gamma A_k + bB_{k+1} \leq q(A_k + \gamma A_{k-1} + bB_k) \leq \ldots \leq q^{k+1}(A_0 + \gamma A_{-1} + bB_0) \leq q^{k+1}((1+\gamma)A_0 + bB_0).$$

Or $A_k \geq 0$ et $B_{k+1} \geq 0$ alors,

$$A_{k+1} \leq q^{k+1}((1 + q - a_1)A_0 + bB_0)$$

De plus, comme $A_k$, $B_k$, $\gamma$ et $b$ sont positifs alors,

$$0 \leq \gamma A_0 + bB_0 = (q - a_1 + 1 - 1)A_0 + bB_0$$

$$\Leftrightarrow A_0 \leq ((1 + q - a_1)A_0 + bB_0) = q^0((1 + q - a_1)A_0 + bB_0)$$

Ainsi,

$$\forall k \in \mathbb{N}, A_k \leq q^k((1 + q - a_1)A_0 + bB_0)$$

$\square$

**Lemme 4.2 :** Si $f \in S_{1,1}^{\mu,L}$, alors $\forall x, y \in \mathbb{R}^n$,

$$f(y) \leq f(x) + \langle \nabla f(x), y - x \rangle + \tfrac{1}{2\mu}||\nabla f(x) - \nabla f(y)||^2$$

$$\text{et } \langle \nabla f(x) - \nabla f(y), x - y \rangle \leq \tfrac{1}{\mu}||\nabla f(x) - \nabla f(y)||^2$$

**Preuve :** Soit $x \in \mathbb{R}^n$.
On considère $\phi(y) = f(y) - \langle \nabla f(x), y \rangle \in S_{1,1}^{\mu,L}$ donc,

$$\nabla \phi(y) = \nabla f(y) - \nabla f(x)$$

On pose, y=x

$$\nabla \phi(x) = \nabla f(x) - \nabla f(x) = 0$$

Donc, $\phi(x) = \min\limits_{v \in \mathbb{R}^n} \phi(v)$, comme $\phi \in S_{1,1}^{\mu,L}$, alors

$$\phi(x) = \min_{v \in \mathbb{R}^n} \phi(v) \geq \min_{v \in \mathbb{R}^n} (\phi(y) + \langle \nabla \phi(y), v - y \rangle + \frac{\mu}{2}||v - y||^2)$$

On pose $g(v) = \phi(y) + \langle \nabla \phi(y), v - y \rangle + \frac{\mu}{2}||v - y||^2$ donc, $g'(v) = \nabla \phi(y) + \mu(v - y) = 0$

$$\Leftrightarrow v = y - \tfrac{1}{\mu} \nabla \phi(y)$$

Ainsi, $\min\limits_{v \in \mathbb{R}^n} g(v) = g(y - \dfrac{1}{\mu} \nabla \phi(y))$ et

$$\begin{aligned}
\phi(x) &\geq \phi(y) + \langle \nabla \phi(y), y - \tfrac{1}{\mu} \nabla \phi(y) - y \rangle + \tfrac{\mu}{2}||y - \tfrac{1}{\mu} \nabla \phi(y) - y||^2 \\
&\geq \phi(y) - \tfrac{1}{\mu}||\nabla \phi(y)||^2 + \tfrac{1}{2\mu}||\nabla \phi(y)||^2 \\
&\geq \phi(y) - \tfrac{1}{\mu}||\nabla \phi(y)||^2 + \tfrac{1}{2\mu}||\nabla \phi(y)||^2 \\
&\geq \phi(y) - \tfrac{1}{2\mu}||\nabla \phi(y)||^2
\end{aligned}$$

On remplace $\phi$

$$f(x) - \langle \nabla f(x), x \rangle \geq f(y) - \langle \nabla f(x), y \rangle - \tfrac{1}{2\mu}||\nabla f(y) - \nabla f(x)||^2$$

$$\Leftrightarrow f(y) \leq f(x) - \langle \nabla f(x), x \rangle + \langle \nabla f(x), y \rangle + \tfrac{1}{2\mu}||\nabla f(y) - \nabla f(x)||^2$$

Ainsi,

$$f(y) \leq f(x) + \langle \nabla f(x), y - x \rangle + \tfrac{1}{2\mu}||\nabla f(x) - \nabla f(y)||^2$$

En inversant x et y, on a

$$f(x) \leq f(y) + \langle \nabla f(y), x - y \rangle + \tfrac{1}{2\mu}||\nabla f(y) - \nabla f(x)||^2$$

On somme les deux inégalitées,

$$f(y)+f(x)\leq$$
$$f(y)+\langle\nabla f(y),x-y\rangle+\tfrac{1}{2\mu}||\nabla f(y)-\nabla f(x)||^2+f(x)+\langle\nabla f(x),y-x\rangle+\tfrac{1}{2\mu}||\nabla f(x)-\nabla f(y)||^2$$

$$\Leftrightarrow 0\leq\langle\nabla f(x)-\nabla f(y),y-x\rangle+\tfrac{1}{\mu}||\nabla f(x)-\nabla f(y)||^2$$

$$\Leftrightarrow\langle\nabla f(x)-\nabla f(y),x-y\rangle\leq\tfrac{1}{\mu}||\nabla f(x)-\nabla f(y)||^2$$

□

**Preuve du théorème 4.1 :** L'itération d'Heavy-ball est $x_{k+1}=x_k-\alpha\nabla f(x_k)+\beta(x_k-x_{k-1}$ donc,

$$||x_{k+1}-x_k||^2=||-\alpha\nabla f(x_k)+\beta(x_k-x_{k-1}||^2$$

$$=\alpha^2||\nabla f(x_k)||^2-2\alpha\beta\langle\nabla f(x_k),x_k-x_{k-1}\rangle+\beta^2||x_k-x_{k-1}||^2$$

De plus, comme $f\in\mathcal{F}_L^{1,1}$, d'après le lemme 3.1, on a

$$0\leq f(y)-f(x)-\langle\nabla f(x),y-x\rangle\leq\tfrac{L}{2}\|x-y\|^2$$

On pose $x=x_k$ et $y=x_{k+1}$ donc,

$$0\leq f(x_{k+1})-f(x_k)-\langle\nabla f(x_k),x_{k+1}-x_k\rangle\leq\tfrac{L}{2}\|x_k-x_{k+1}\|^2,$$

$$0\leq f(x_{k+1})-f(x_k)-\langle\nabla f(x_k),-\alpha\nabla f(x_k)+\beta(x_k-x_{k-1}\rangle\leq\tfrac{L}{2}\|x_k-x_{k+1}\|^2,$$

$$0\leq f(x_{k+1})-f(x_k)+\alpha||\nabla f(x_k)||^2-\beta\langle\nabla f(x_k),x_k-x_{k-1}\rangle\leq\tfrac{L}{2}\|x_k-x_{k+1}\|^2,$$

or,

$$\tfrac{L}{2}||x_{k+1}-x_k||^2=\tfrac{L}{2}(\alpha^2||\nabla f(x_k)||^2-2\alpha\beta\langle\nabla f(x_k),x_k-x_{k-1}\rangle+\beta^2||x_k-x_{k-1}||^2)$$

$$=\tfrac{L\alpha^2}{2}||\nabla f(x_k)||^2-\tfrac{L\alpha\beta^2}{2}\langle\nabla f(x_k),x_k-x_{k-1}\rangle+\tfrac{L\alpha\beta^2}{2}||x_k-x_{k-1}||^2$$

donc,

$$f(x_{k+1})-f(x_k)+\alpha||\nabla f(x_k)||^2-\beta\langle\nabla f(x_k),x_k-x_{k-1}\rangle\leq$$
$$\tfrac{L\alpha^2}{2}||\nabla f(x_k)||^2-\tfrac{L\alpha\beta^2}{2}\langle\nabla f(x_k),x_k-x_{k-1}\rangle+\tfrac{L\alpha\beta^2}{2}||x_k-x_{k-1}||^2,$$

$$\Leftrightarrow$$

$$f(x_{k+1})-f^*\leq f(x_k)-f^*-\alpha(1-\alpha L^2)^k\|\nabla f(x_k)\|^2+\tfrac{L\beta^2}{2}\|x_k-x_{k-1}\|^2+\beta(1-\alpha L)h\langle\nabla f(x_k),x_k-x_{k-1}\rangle.$$

On a,

$$\tfrac{L\theta}{2(1-\theta)}||x_{k+1}-x_k||^2=\tfrac{L\theta\alpha^2}{2(1-\theta)}||\nabla f(x_k)||^2+\tfrac{L\theta\beta^2}{2(1-\theta)}||x_k-x_{k-1}||^2-\tfrac{2\alpha\beta L}{2(1-\theta)}\langle\nabla f(x_k),x_k-x_{k-1}\rangle$$

donc,

$$f(x_{k+1}) - f^* + \frac{L\theta}{2(1-\theta)}||x_{k+1} - x_k||^2 \leq f(x_k) - f^* + (\frac{L\theta\alpha^2}{2(1-\theta)} - \alpha(1 - \frac{\alpha L}{2})\|\nabla f(x_k)\|^2 + (\frac{\beta^2 L\theta}{2(1-\theta)} + \frac{L\beta^2}{2})\|x_k - x_{k-1}\|^2 + (\frac{-2\alpha\beta L\theta}{2(1-\theta)} + \beta(1 - \alpha L))\langle \nabla f(x_k), x_k - x_{k-1}\rangle.$$

$$\leq f(x_k) - f^* + \alpha(\frac{L\alpha}{2(1-\theta)} - 1)\|\nabla f(x_k)\|^2 + \frac{\beta^2 L}{2(1-\theta)}\|x_k - x_{k-1}\|^2 + \beta(1 - \frac{\alpha L}{1-\theta})\langle \nabla f(x_k), x_k - x_{k-1}\rangle.$$

Supposons que $\frac{1-\theta}{L} \leq \alpha \leq \frac{2(1-\theta)}{L}$, comme $f \in \mathcal{S}_{1,1}^{\mu,L}$ alors d'après le lemme 4.2, $\forall x, y \in \mathbb{R}^n$

$$f(y) \leq f(x) + \langle \nabla f(x), y - x\rangle + \frac{1}{2\mu}||\nabla f(x) - \nabla f(y)||^2$$

si on pose $x = x_k$ et $y = x_{k-1}$,

$$\langle \nabla f(x), x_k - x_{k-1}\rangle \leq f(x_k) - f(x_{k-1}) + \frac{1}{2\mu}||\nabla f(x_k) - \nabla f(x_{k-1})||^2$$

$$\leq f(x_k) - f(x_{k-1}) + \frac{\mu^2}{2\mu}||x_k - x_{k-1}||^2 \text{ car gradient de f lipschitz}$$

$$\leq f(x_k) - f(x_{k-1}) + \frac{\mu}{2}||x_k - x_{k-1}||^2$$

si on pose $x = x^*$ et $y = x_k$

$$f(x_k) \leq f(x^*) + \langle \nabla f(x^*), x_k - x^*\rangle + \frac{1}{2\mu}||\nabla f(x^*) - \nabla f(x_k)||^2$$

$$\Leftrightarrow f(x_k) - f^* \leq \frac{1}{2\mu}||\nabla f(x_k)||^2$$

$$\Leftrightarrow 2\mu(f(x_k) - f^*) \leq ||\nabla f(x_k)||^2$$

donc,

$$f(x_{k+1}) - f^* + \frac{L\theta}{2(1-\theta)}||x_{k+1} - x_k||^2 \leq f(x_k) - f^* + 2\alpha\mu(\frac{L\alpha}{2(1-\theta)} - 1)(f(x_k) - f^*) + \frac{\beta^2 L}{2(1-\theta)}\|x_k - x_{k-1}\|^2 + \beta(1 - \frac{\alpha L}{1-\theta})(f(x_k) - f(x_{k-1}) + \frac{\beta\mu}{2}(1 - \frac{\alpha L}{1-\theta})\|x_k - x_{k-1}\|^2$$

$$\leq (1 + \frac{\alpha^2\mu L}{1-\theta} - 2\alpha\mu + \beta - \frac{\beta\alpha L}{1-\theta})f(x_k) + \frac{\beta^2 L}{2(1-\theta)}\|x_k - x_{k-1}\|^2 - (1 + \frac{\alpha^2\mu L}{1-\theta} - 2\alpha\mu)f^* + \frac{\beta\mu}{2}(1 - \frac{\alpha L}{1-\theta})\|x_k - x_{k-1}\|^2 + (\frac{\alpha L\beta}{1-\theta} - \beta)f(x_{k-1})$$

$$\leq (1 + \frac{\alpha^2\mu L}{1-\theta} - 2\alpha\mu + \beta - \frac{\beta\alpha L}{1-\theta})(f(x_k) - f^*) + \frac{\beta^2 L}{2(1-\theta)}\|x_k - x_{k-1}\|^2 - (\frac{\beta\alpha L}{1-\theta} - \beta)f^* + \frac{\beta\mu}{2}(1 - \frac{\alpha L}{1-\theta})\|x_k - x_{k-1}\|^2 + (\frac{\alpha L\beta}{1-\theta} - \beta)f(x_{k-1})$$

Ainsi,

$$f(x_{k+1}) - f^* + \frac{L\theta}{2(1-\theta)}||x_{k+1} - x_k||^2 \leq (1 - 2\alpha\mu(1 - \frac{\alpha L}{2(1-\theta)}) - \beta(\frac{\alpha L}{1-\theta} - 1))(f(x_k) - f^*) + \beta(\frac{\alpha L}{1-\theta} - 1)(f(x_{k-1}) - f^*) + \frac{\beta}{2}(\mu(1 - \frac{\alpha L}{1-\theta}) + \frac{L\beta}{1-\theta})||x_k - x_{k-1}||^2$$

On pose, $b = \frac{L\theta}{2(1-\theta)}$, $a_1 = 1 - 2\alpha\mu(1 - \frac{\alpha L}{2(1-\theta)}) - \beta(\frac{\alpha L}{1-\theta} - 1)$, $a_2 = \beta(\frac{\alpha L}{1-\theta} - 1)$, $c = \frac{\beta}{2}(\mu(1 - \frac{\alpha L}{1-\theta}) + \frac{L\beta}{1-\theta})$, $A_k = f(x_k) - f^*$ et $B_k = ||x_k - x_{k-1}||$ donc,

$$A_{k+1} + bB_{k+1} \leq a_1 A_k + a_2 A_{k-1} + cB_k$$

De plus, comme $\theta \in ]0, 1[$ et $\frac{1-\theta}{L} \leq \alpha < \frac{2(1-\theta)}{L}$ et $L > 0$ alors,

$$b = \frac{L\theta}{2(1-\theta)} > 0 \text{ et}$$

$$\frac{1-\theta}{L} \leq \alpha < \frac{2(1-\theta)}{L}$$

$$\Leftrightarrow -\frac{1}{2} \geq \frac{-\alpha L}{2(1-\theta)} > -1$$

$$\Leftrightarrow \frac{\mu}{2} \geq \mu(1 - \frac{\alpha L}{2(1-\theta)}) > 0$$

$$\Leftrightarrow -\frac{\mu(1-\theta)}{L} \leq -2\alpha\mu(1 - \frac{\alpha L}{2(1-\theta)}) < 0$$

$$\Leftrightarrow 1 - \frac{\mu(1-\theta)}{L} \leq 1 - 2\alpha\mu(1 - \frac{\alpha L}{2(1-\theta)}) < 1$$

$$\Leftrightarrow 1 - \frac{\mu(1-\theta)}{L} \leq a_1 + a_2 < 1$$

d'où, $a_1 + a_2 < 1$. On veut aussi que $c < b$

$$\Leftrightarrow \frac{\beta}{2}(\mu(1 - \frac{\alpha L}{1-\theta}) + \frac{L\beta}{1-\theta}) < \frac{L\theta}{2(1-\theta)}$$

$$\Leftrightarrow \beta^2 \frac{L}{2(1-\theta)} + \beta\frac{\mu}{2}(1 - \frac{\alpha L}{1-\theta}) - \frac{L\theta}{2(1-\theta)} < 0,$$

$$\Delta = (\frac{\mu}{2}(1 - \frac{\alpha L}{1-\theta}))^2 - 4 * \frac{L}{2(1-\theta)} * (-\frac{L\theta}{2(1-\theta)})$$

$$= \frac{\mu^2}{4}(1 - \frac{2\alpha L}{1-\theta} + \frac{\alpha^2 L^2}{(1-\theta)^2}) + \frac{L^2\theta}{(1-\theta)^2}$$

$$= \frac{\mu^2(1-\theta)^2 - 2\alpha L\mu^2(1-\theta) + \mu^2\alpha^2 L^2 + 4L^2\theta}{4(1-\theta)^2}$$

$$= \frac{1}{4(1-\theta)^2}(\mu^2(\alpha L + \theta - 1)^2 + 4\theta L^2) > 0,$$

$$\beta_1 = \frac{-\frac{\mu}{2}(1 - \frac{\alpha L}{1-\theta}) - \sqrt{\frac{1}{4(1-\theta)^2}(\mu^2(\alpha L + \theta - 1)^2 + 4\theta L^2)}}{2 * \frac{L}{2(1-\theta)}}$$

$$= -\frac{\mu(1-\theta)}{2L}(1 - \frac{\alpha L}{1-\theta}) - \sqrt{\frac{1}{4L^2}(\mu^2(\alpha L + \theta - 1)^2 + 4\theta L^2)}$$

$$= \frac{1}{2}(\frac{\mu}{L}(\alpha L + \theta - 1) - \sqrt{\frac{\mu^2}{L^2}(\alpha L + \theta - 1)^2 + 4\theta}) < 0$$

$$\text{et } \beta_2 = \frac{1}{2}(\frac{\mu}{L}(\alpha L + \theta - 1) + \sqrt{\frac{\mu^2}{L^2}(\alpha L + \theta - 1)^2 + 4\theta}) \geq 0$$

donc $\beta \in [0, \frac{1}{2}(\frac{\mu}{L}(\alpha L + \theta - 1) + \sqrt{\frac{\mu^2}{L^2}(\alpha L + \theta - 1)^2 + 4\theta})[$.
On pose $\lambda = 1 - \theta$ donc, $\lambda \in ]0, 1[$ et $\frac{\lambda}{2} \leq \alpha \leq \frac{2\lambda}{L}$

$$\text{ie } \alpha \in ]0, \frac{2}{L}[,\ \lambda \in ]\frac{\alpha L}{2}, \min(\alpha L, 1)[ \text{ et } 0 \leq \beta \leq \frac{1}{2}(\frac{\mu}{L}(\alpha L - \lambda) + \sqrt{\frac{\mu^2}{L^2}(\alpha L - \lambda)^2 + 4(1-\lambda)})[$$

On pose $\lambda = \frac{\alpha L}{2}$ d'où $0 \leq \beta \leq \frac{1}{2}(\frac{\mu\alpha}{2} + \sqrt{\frac{\mu^2\alpha^2}{4} + 4(1 - \frac{\alpha L}{2})})[$
On peut alors appliquer le lemme 4.1 et on a,

$$A_k \leq q^k((q - a_1 + 1)A_0 + bB_0)$$

$$\Leftrightarrow f(x_k) - f* \leq q^k((q - a_1 + 1)(f(x_0) - f^*) + b||x_0 - x_0|^2)$$

$$\leq q^k(q - a_1 + 1)(f(x_0) - f^*)$$

$$\leq q^k h(f(x_0) - f^*) \text{ avec } h = q - a_1 + 1$$

$\square$

Le théorème énonce que si certaines conditions sont remplies, à savoir que le paramètre $\alpha$ utilisé dans l'algorithme est choisi dans l'intervalle $]0, \frac{2}{L}[$ et que le paramètre $\beta$ satisfait $0 \leq \beta \leq \frac{1}{2}(\frac{\mu\alpha}{2} + \sqrt{\frac{\mu^2\alpha^2}{4} + 4(1 - \frac{\alpha L}{2})})[$.
La vitesse de convergence est exponentielle car $q^k = e^{k \ln(q)} = O(e^k)$.

Le théorème 4.1 est un cas du théorème 3.1 car dans le théoréme 4.1, il faut que la fonction soit fortement convexe et à gradient Lipschitz alors que dans le théorème 3.1, il suffit que la fonction soit fortement convexe. En revanche, la convergence dans le théorème 4.1 est plus rapide car elle est exponentielle alors que dans le théorème 3.1 elle est linéaire.

Dans la parie suivante, nous allons comparer numériquement les deux algorithmes à travers l'exemple de deux fonctions quadratiques.

# 5 Comparaison numérique avec la descente de gradient standard

Nous allons comparer numériquement l'algorithme de descente de gradient standard à pas fixe avec l'algorithme de descente de gradient avec moment à pas fixe. Tout d'abord nous allons implémenter les deux algorithmes sur python.

L'algorithme de descente de gradient standard à pas fixe :

```
def desc_gradient_pas_fixe(x0,eps,f,gradf,t,IterMax=10000):
    x = x0
    Xk = [x0]
    k = 0
    fx = f(x)
    FXk = [fx]
    gf = gradf(x)
    normgf2 = np.dot(gf.flatten(),gf.flatten())
    while (normgf2 > eps**2 and k < IterMax):
        x = x - t*gf
        fx = f(x)
        Xk.append(x)
        FXk.append(fx)
        k = k+1
        gf = gradf(x)
        normgf2 = np.dot(gf.flatten(),gf.flatten())
    return Xk, FXk
```

L'algorithme de descente de gradient standard à pas fixe avec moment :

```
def desc_gradient_pas_fixe_avec_moment(x0,eps,f,gradf,alpha,beta,IterMax=10000):
    x = x0
    x_=x0
    Xk = [x0]
    k = 0
    fx = f(x)
    FXk = [fx]
    gf = gradf(x)
    normgf2 = np.dot(gf.flatten(),gf.flatten())
    while (normgf2 > eps**2 and k < IterMax):
        x = x - alpha*gf+beta*(x-x_)
        x_=Xk[-1]
        fx = f(x)
        Xk.append(x)
        FXk.append(fx)
        k = k+1
        gf = gradf(x)
        normgf2 = np.dot(gf.flatten(),gf.flatten())
    return Xk, FXk
```

La première fonction que nous allons comparer est :

$$f : \mathbb{R}^2 \to \mathbb{R} \text{ définie par } f(x) = \frac{1}{2}\langle Ax, x\rangle - \langle b, x\rangle + c$$

où $A = \begin{bmatrix} 4 & 1 \\ 1 & 3 \end{bmatrix}$ , $b = \begin{bmatrix} -1 \\ 1 \end{bmatrix}$ et $c = 9$
$A$ est une matrice carrée symétrique donc f est bien une fonction quadratique.

On calcul, $x^t Ax = (x_1+x_2)^2+3x_2^2+2x_2^2 > 0$ donc A est une matrice définie positive symétrique, une solution optimale $x^*$ éxiste où $x^* = A^{-1}b$

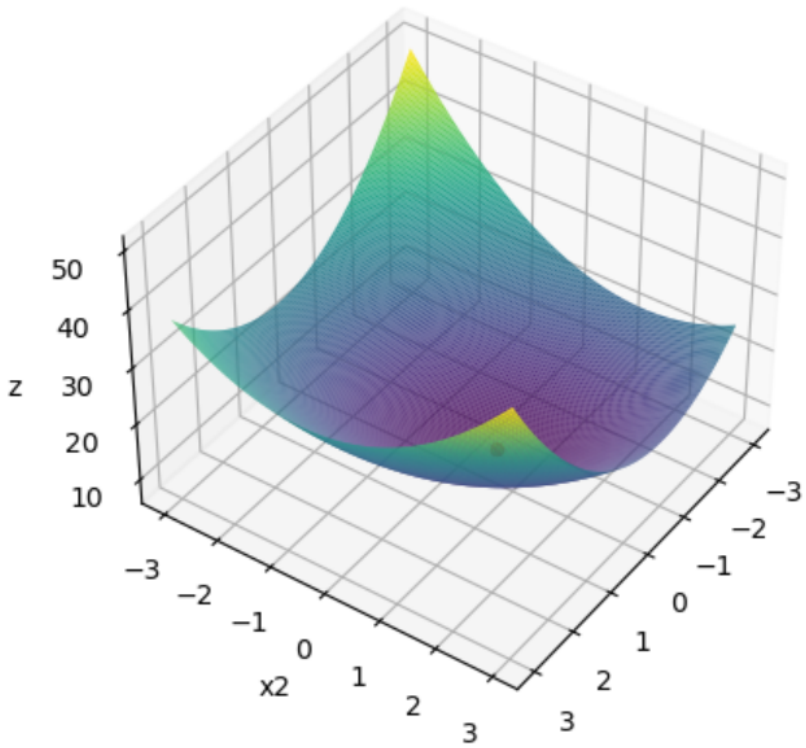
Calculons la solution exacte sur python :

```
A=np.array([[4,1],[1,3]])
b=np.transpose(np.array([-1,1]))
c=9

def f(x):
    Ax = np.transpose(A@x)
    return 0.5*np.dot(Ax.flatten(),x.flatten()) - np.dot(b.flatten(),x.flatten()) + c

def gradf(x):
    return A@x - b

#calcul de la solution exacte
xstar = np.linalg.solve(A,b)
zstar = f(xstar)
print("xstar = ",xstar)
print("f(xstar) = ",zstar)
print("gradf(xstar) = ",gradf(xstar))
```

```
xstar =  [-0.36363636  0.45454545]
f(xstar) =  8.590909090909092
gradf(xstar) =  [ 0.00000000e+00 -1.11022302e-16]
```

Visualisons le graphique de f avec la solution :

```
#tracé de f sur ]-10,10[^2
N=200
x1=np.linspace(-3,3,N)
x2=x1
X1, X2 = np.meshgrid(x1,x2)

def f_int(x1,x2):
    return f(np.array([x1,x2]))
fv = np.vectorize(f_int)

Y = fv(X1, X2)

fig = plt.figure()
ax = plt.axes(projection='3d')
ax.plot_surface(X1,X2,Y,rstride=1,cstride=1,cmap='viridis',edgecolor='none');
ax.scatter(xstar[0],xstar[1],zstar,marker="o", c="red")
ax.view_init(40,35) #elevation (above the x-y plane) in degree and azimuth (rotate 35ax.set_xlabel('x1')
ax.set_ylabel('x2')
ax.set_zlabel('z');
```




Maintenant, calculons les solutions. Pour le choix des paramètres alpha, beta et t, nous allons prendre les choix optimaux qui sont calculés dans l'artcile Convergence of Heavy-Ball Method and Nesterov's Accelerated Gradient on Quadratic Optimization par Trung Vu. C'est à dire :

$t = \frac{2}{L+l}$, $\alpha = (\frac{2}{\sqrt{L}+\sqrt{l}})^2$ et $\beta = \frac{\sqrt{L}-\sqrt{l}}{\sqrt{L}+\sqrt{l}}$ où $L$ est la plus grande valeur propre de $A$ et $l$ la plus petite.

Calculons ces valeurs propres,

$$\begin{aligned}\mathrm{Det}(A-\lambda I) &= \begin{vmatrix} 4-\lambda & 1 \\ 1 & 3-\lambda \end{vmatrix} \\ &= (4-\lambda)(3-\lambda) - 1\cdot 1 \\ &= \lambda^2 - 7\lambda + 11\end{aligned}$$

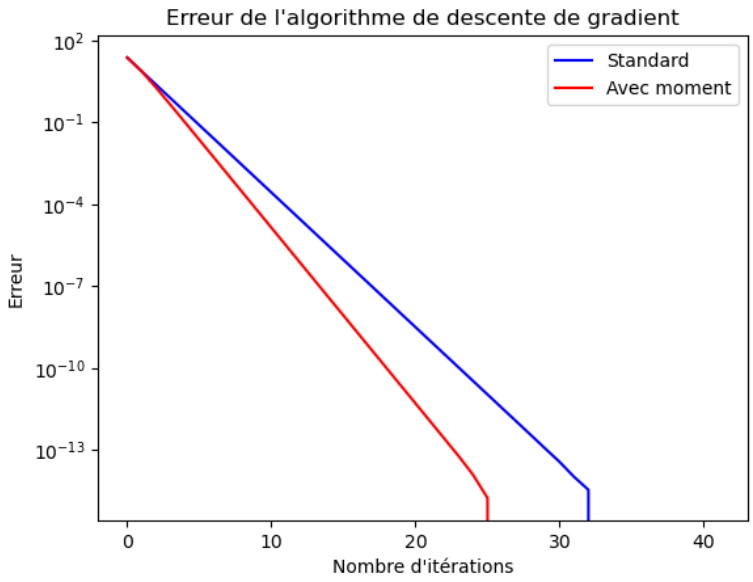
Les valeurs propres sont les solutions de l'équation $\lambda^2 - 7\lambda + 11 = 0$. En résolvant cette équation, on trouve que les valeurs propres sont $L \approx 6.37$ et $\lambda_2 l 4.63$.

```
L=6.37 # plus grande valeur propre de A
l=4.63 # plus petite valeur propre de A

t=2/(L+l)
alpha=4/(np.sqrt(L)+np.sqrt(l))**2
beta=(np.sqrt(L)-np.sqrt(l))/(np.sqrt(L)+np.sqrt(l))

x0 = np.array([-3, 4])
eps = 1.e-9

Xks, FXks = desc_gradient_pas_fixe(x0,eps,f,gradf,t,IterMax=10000)
Xkm,FXkm = desc_gradient_pas_fixe_avec_moment(x0,eps,f,gradf,alpha,beta,IterMax=10000)

print('La solution calculée avec l algorithme de descente de gradient standard à pas fixe est', Xks[-1] ,
      'avec un nombre d itérations de', len(Xks))
print('La solution calculée avec l algorithme de descente de gradient avec moment à pas fixe est', Xkm[-1] ,
      'avec un nombre d itérations de', len(Xkm))
```

```
La solution calculée avec l algorithme de descente de gradient standard à pas fixe est [-0.36363636  0.45454545] a
d itérations de 42
La solution calculée avec l algorithme de descente de gradient avec moment à pas fixe est [-0.36363636  0.4545454
bre d itérations de 33
```

On voit que les deux algogithmes ont bien fonctionnés, ils ont la même solution que nous avons calculer au début. En revanche, on voit que l'algorithme de descente de gradient à pas fixe avec moment a besoin de moins d'itérations, 33 contre 42 pour l'algorithme de descente de gradient standard à pas fixe donc l'algorithme de descente de gradient avec moment est bien plus rapide. Comparons l'évolution de l'erreur en fonction du nombres d'itérations des algorithmes :

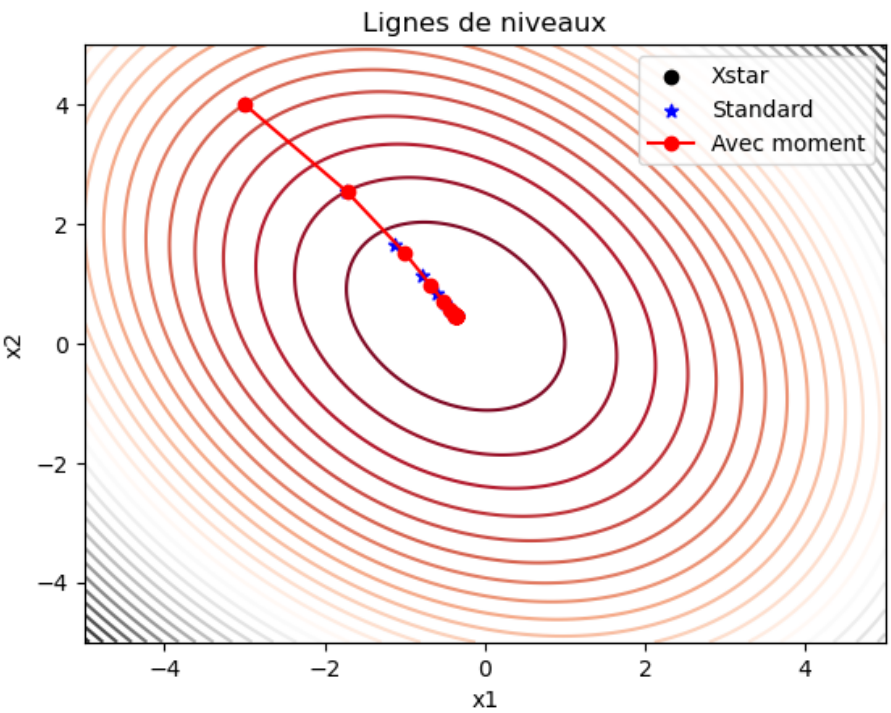
```python
# evolution de l'erreur pour l'algorithme standard
x1ks = [Xks[i][0] for i in range(len(Xks))]
x2ks = [Xks[i][1] for i in range(len(Xks))]
errFs = [FXks[i]-zstar for i in range(len(FXks))]

# evolution de l'erreur pour l'algorithme avec moment
x1km = [Xkm[i][0] for i in range(len(Xkm))]
x2km = [Xkm[i][1] for i in range(len(Xkm))]
errFm = [FXkm[i]-zstar for i in range(len(FXkm))]

plt.plot(errFs, color='blue')
plt.plot(errFm, color='red')
plt.yscale('log')

# Titre du graphique
plt.title("Erreur de l'algorithme de descente de gradient")

# Légendes des courbes
plt.legend(['Standard', 'Avec moment'])

# Étiquettes des axes
plt.xlabel("Nombre d'itérations")
plt.ylabel("Erreur")

# Afficher le graphique
plt.show()
```




On voit que l'algorithme de descente de gradient standard a une plus petite erreur que l'algorithme de descente de gradient avec moment pour chaques itérations. Ainsi l'algorithme de descente de gradient avec moment est meilleure que le standard.

Pour finir avec cette exemple regardons les lignes de niveaux des deux algorithmes.

```python
# Calul de la fonction sur une grille de [-5,5]x[-5,5]
N = 200
x1 = np.linspace(-5,5,N)
x2 = np.linspace(-5,5,N)
X1, X2 = np.meshgrid(x1,x2)

def f_int(x1,x2):
    return f(np.array([[x1],[x2]]))

fv = np.vectorize(f_int)
Z = fv(X1,X2)

plt.figure()
# tracé des lignes de niveaux
plt.contour(X1, X2, Z, 30, cmap='RdGy')
plt.scatter(xstar[0],xstar[1],marker="o", c="black")
plt.scatter(x1ks,x2ks,marker="*",color="blue")
plt.plot(x1km,x2km,marker="o",color="red")

# Titre du graphique
plt.title("Lignes de niveaux")

# Légendes des courbes
plt.legend(['Xstar', 'Standard', 'Avec moment'])

# Étiquettes des axes
plt.xlabel("x1")
plt.ylabel("x2")

# Afficher le graphique
plt.show()
```




On voit que pour $x_0$ et $x_1$, les deux points des deux algorithmes se superposent, c'est parce que les deux $x_0$ sont égaux, et lors de la première itération de l'algorithme de gradient à pas fixe

avec moment, on a $x_1 = x_0 - \alpha\nabla f(x_0) + \beta(x_0 - x_{-1}) = x_0 - \alpha\nabla f(x_0)$ car $x_{-1} = x_0$. Donc pour la première itération de l'algorithme de descente de gradient avec moment, on a pas de moment. Ensuite, on voit que pour chaques itérations l'algorithme de descente de gradient à pas fixe avec moment va un peu plus loin. Dès la troisième itération, on voit bien que les deux points ne sont pas supérposé et le point de l'algorithme de descente de gradient à pas fixe avec moment est plus proche de xstar que l'algorithme de descente de gradient à pas fixe. Ainsi les lignes de niveaux confirment que l'algorithme de descente de gradient à pas fixe avec moment converge plus vite.

La seconde fonction que nous allons comparer est l'équation de poisson en dimension 100 :

$$f : \mathbb{R}^{100} \to \mathbb{R} \text{ définie par } f(x) = \tfrac{1}{2}\langle Ax, x\rangle - \langle b, x\rangle + c$$

où

```
n = 100
A = (2*(np.eye(n))-np.eye(n,k=1)-np.eye(n,k=-1))*(n+1)**2
b = -np.ones((n,1))
```

et le vecteur $b$ constant à la valeur $-1$ (qui correspond à la discrétisation par différences finies d'un problème de Poisson 1D $u'' = b$ pour $u \in \mathcal{C}^2(]0,1[)$)

On calcul les valeurs propres de A sur python :

```
eigenvalues = np.linalg.eigvals(A)  # Calcul des valeurs propres
L=np.max(eigenvalues)
l=np.min(eigenvalues)
print(L,l)
```

```
40794.131191321234 9.868808678857832
```

La plus petite valeure prorpre est $l = 9.87$ et la plus grande est $L = 40794.13$. On calcul la solution exacte sur python :

```
# fonctionnelle quadratique
def f(x):
    #return 0.5*(np.dot(A@x,x)) - np.dot(b,x) + c # ne fonctionne pas toujours si np.
    Ax = A@x
    return 0.5*np.dot(Ax.flatten(),x.flatten()) - np.dot(b.flatten(),x.flatten()) + c

def gradf(x):
    return A@x - b

xstar = np.linalg.solve(A,b)
zstar = f(xstar)
print("valeur minimale = ",zstar)
```

```
valeur minimale =  -4.20792079207921
```

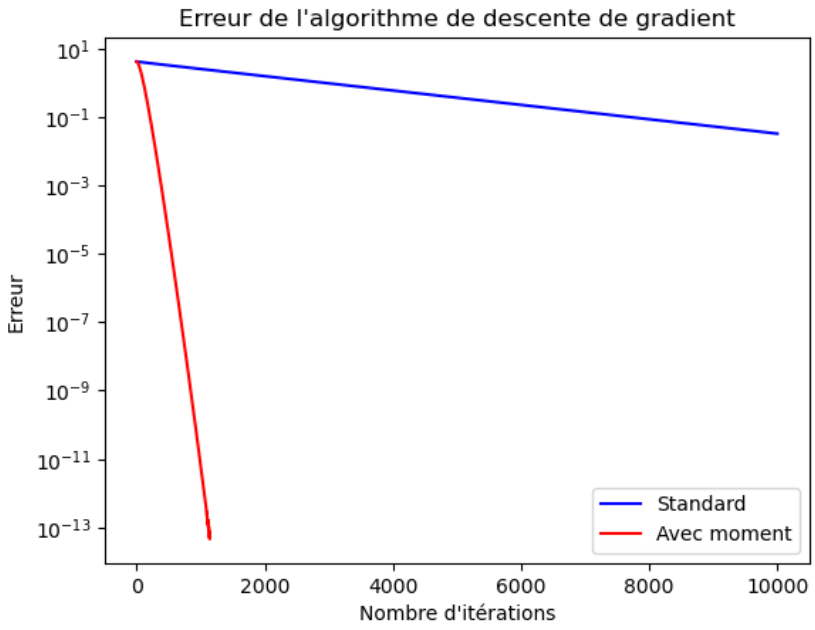
Maintenant, calculons la solution avec les deux algorithmes

```python
x0 = np.zeros((n,1))
eps = 1.e-6

t=1/L
alpha=1/L
beta=(np.sqrt(L)-np.sqrt(l))/(np.sqrt(L)+np.sqrt(l))


Xks, FXks = desc_gradient_pas_fixe(x0,eps,f,gradf,t)
Xkm,FXkm = desc_gradient_pas_fixe_avec_moment(x0,eps,f,gradf,alpha,beta)

print('La solution calculée avec l algorithme de descente de gradient standard à pas fixe est', FXks[-1] ,
      'avec un nombre d itérations de', len(Xks))
print('La solution calculée avec l algorithme de descente de gradient avec moment à pas fixe est', FXkm[-1] ,
      'avec un nombre d itérations de', len(Xkm))
```

```
La solution calculée avec l algorithme de descente de gradient standard à pas fixe est -4.175094251006875 avec un nombre d itér
ations de 10001
La solution calculée avec l algorithme de descente de gradient avec moment à pas fixe est -4.207920792079149 avec un nombre d i
térations de 1144
```

On voit que l'algorithme de descente de gradient à pas fixe avec moment est beaucoup plus proche de la solution et a besoin de moins d'itérations que l'algorithme de descente de gradient standard à pas fixe.

Regardons l'evolution de l'erreur en fonction du nombres d'itérations



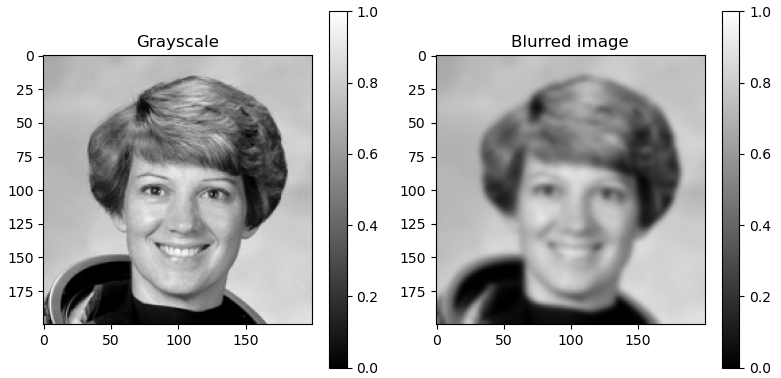

On voit bien que l'algorithme de descente de gradient à pas fixe avec moment est meilleur, pour chaques itérations, il a une plus petite erreur et a besoin de seulement 1144 itérations alors que l'algortihme de descente de gradient standard à pas fixe va jusqu'au maximum de nombres d'itérations fixé qui est 10001 et malgrès cela son erreur reste toujours grande.

# 6 Comparaison numérique avec la descente de gradient standard sur un problème de restauration d'images : Déflouage par régularisation de Tichonov (en python)

Dans cette partie nous allons utiliser l'algorithme de descente de gradient pour déflouter une image flou à l'aide de la méthode de régularisation de Tichonov. Nous allons alors comparer la pérformance des deux algorithmes dans ce problème.

Tout d'abord, on prend une image et on l'a rend flou en ajoutant un bruit gaussien :

```
# Define Gaussian blur operator:
sigblur = 2.
s = int(3*sigblur)
w = 2*s+1
kernel = np.zeros(w)
for t in np.arange(w):
    kernel[t] = np.exp(-(t-s)**2/(2*sigblur**2))
kernel /= sum(kernel)
gausskernel = np.zeros((w,w))
for t1 in np.arange(w):
    for t2 in np.arange(w):
        gausskernel[t1,t2] = kernel[t1]*kernel[t2]
blur = lambda x: scipy.ndimage.convolve(x, gausskernel)

original = skimage.data.astronaut()
imggray = skimage.color.rgb2gray(original)
imggray = imggray[0:200,120:320]
M, N = imggray.shape
imgblurred = blur(imggray)

fig, ax = plt.subplots(1, 2, figsize=(8, 4))
im0 = ax[0].imshow(imggray, cmap=plt.cm.gray)
im0.set_clim(0,1)
ax[0].set_title("Grayscale")
plt.colorbar(im0,ax=ax[0])
im1 = ax[1].imshow(imgblurred, cmap=plt.cm.gray)
im1.set_clim(0,1)
ax[1].set_title("Blurred image")
plt.colorbar(im1,ax=ax[1])
fig.tight_layout()
plt.show()
```



Maintenant qu'on a notre image floutée, on va la déflouter en minimisant cette fonction :

$$f(x) = \frac{1}{2}\|Hx - y\|^2$$

Le gradient de la fonction est donné par :

$$\nabla f(x) = H^T(Hx - y)$$

Le paramètre $L$ est fixé à 1.
Ici, $H : x \mapsto Hx$ représente l'opérateur linéaire qui effectue la convolution discrète 2D avec un noyau gaussien discret, noté $\kappa * x$.

On code les fonctions et les algorithmes sur pyhton :

```
y = imgblurred+4./255.*np.random.randn(M, N) # blur with noise
x0 = y

def f(x):
    return 0.5*np.sum((blur(x)-y)**2)

def gradf(x):
    return blur(blur(x)-y)

def gradient_descent_fixed_step_size(f,gradf,x0,niter,t,verb=True):
    FXk = f(x0)
    x = x0.copy()
    for k in range(niter):
        x = x - t * gradf(x)
        fx = f(x)
        FXk = np.append(FXk,fx)
        if verb:
            print(['it ', k, 'f(x) = ', fx])
    return x, FXk

def gradient_descent_fixed_step_size_with_moment(f,gradf,x0,niter,alpha,beta,verb=True):
    FXk = f(x0)
    x = x0.copy()
    x_ = x0.copy()
    for k in range(niter):
        x,x_ = x -  alpha*gradf(x)+beta*(x-x_),x
        fx = f(x)
        FXk = np.append(FXk,fx)
        if verb:
          print(['it ', k, 'f(x) = ', fx])
    return x, FXk

def imshowlistgray(imgs, titles):
    fig, ax = plt.subplots(1, len(imgs), figsize = (10*len(imgs),10))
    for k, im in enumerate(imgs):
        im = ax[k].imshow(im, cmap = plt.cm.gray)
        im.set_clim(0,1)
        ax[k].set_title(titles[k])
```

Exécutons les deux algorithmes avec un nombre d'itérations de 1000 :

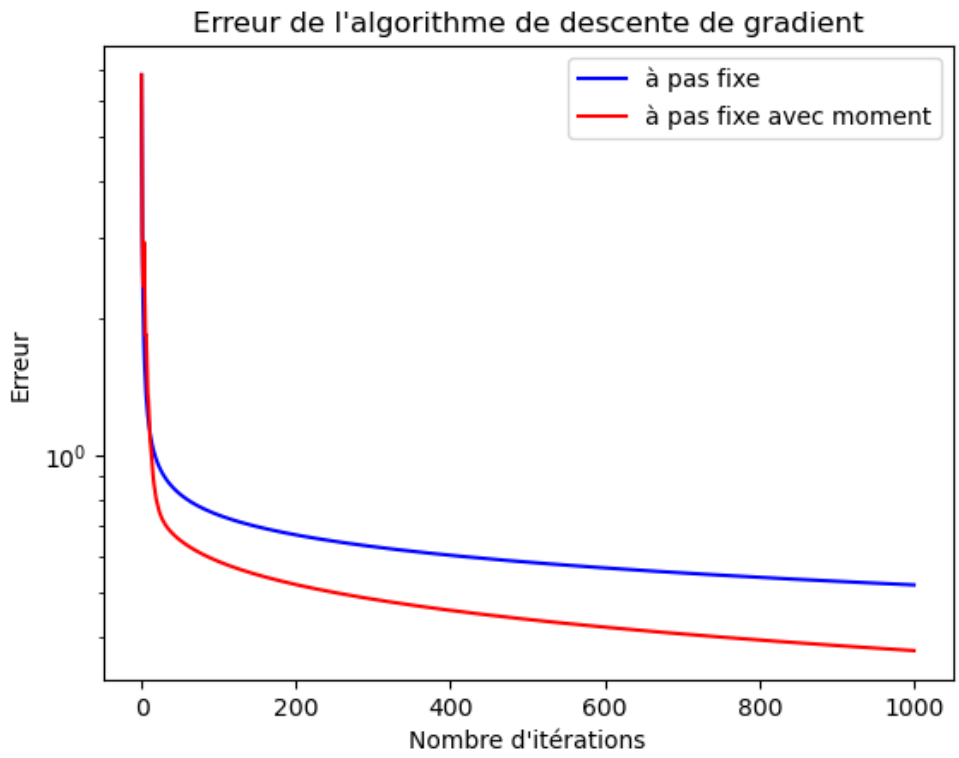
```
L=1
t = 1/L
alpha=1/L
beta=0.8
niter = 1000
x1s, FXk1s = gradient_descent_fixed_step_size(f,gradf,x0,niter,t,verb=False)
x1m, FXk1m = gradient_descent_fixed_step_size_with_moment(f,gradf,x0,niter,alpha,beta,verb=False)


imshowlistgray([imggray, x0, x1s, x1m],['original', 'init', 'pas fixe', 'pas fixe avec moment'])
```

On voit alors que l'image est défloutée en exécutant les deux algorithmes, il reste encore un peu de flou et un bruit apparait . L'image défloutée avec l'algorithme de de descente de gradient à pas fixe avec moment est un peu moins flou et a plus de bruits que l'autre. Pour voir cela, on peut comparer l'erreur des algortihmes.

Pour obtenir xstar, on va exécuter l'algorithme de descente de gradient à pas fixe avec un grand nombre d'itérations (1000x100), l'algortihme va prendre beaucoup plus de temps à s'exécuter.

Comparons l'erreur :

```
errFXk1s=[FXk1s[i]-FXkstar1[-1] for i in range(len(FXk1s))]
errFXk1m=[FXk1m[i]-FXkstar1[-1] for i in range(len(FXk1m))]

plt.plot(errFXk1s, color='blue')
plt.plot(errFXk1m, color='red')
plt.yscale('log')

# Titre du graphique
plt.title("Erreur de l'algorithme de descente de gradient")

# Légendes des courbes
plt.legend(['à pas fixe', 'à pas fixe avec moment'])

# Étiquettes des axes
plt.xlabel("Nombre d'itérations")
plt.ylabel("Erreur")

# Afficher le graphique
plt.show()
```




On voit que la descente de gradient à pas fixe avec moment a une plus petite erreur pour chaques itérations donc il est bien meilleur.

L'algorithme a bien déflouté l'image mais on voit un bruit qui apparait. Pour ne pas avoir ce bruit, on va introduire un terme de régularisation dans la fonction, cette méthode s'appele la régularisaton de Tikhonov :

$$g(x) = f(x) + \tau\frac{1}{2}\|\nabla x\|^2 = \frac{1}{2}\|Hx - y\|^2 + \frac{\tau}{2}\|\nabla x\|^2$$

où $\nabla : \mathbb{R}^{M\times N} \to \left(\mathbb{R}^{M-1\times N-1}\right)^2$ est le gradient discret direct défini par

$$(\nabla x)_{k,l} = (x_{k+1,l} - x_{k,l}, x_{k,l+1} - x_{k,l}) \in \mathbb{R}^2.$$

et le gradient de g est défini par :

$$\nabla g(x) = \nabla f(x) - \Delta x = H^T(Hx - y) - \tau\Delta x,$$

où $\Delta x$ est le Laplacien discret, c'est-à-dire, la convolution de $x$ avec le noyau

$$\begin{pmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{pmatrix}.$$

On code ces fonctions sur python :

```
def d1(x):
    d1= np.array([[-1, 1]])
    return scipy.ndimage.convolve(x, d1)

def d2(x):
    d2= np.array([[1], [-1]])
    return scipy.ndimage.convolve(x, d2)


def grad(x):
    return d2(d1(x))


def g(x):
    return f(x) + tau * 0.5 * np.sum(grad(x)**2)


def gradg(x):
    matx=np.array([[0,1,0],[1,-4,1],[0,1,0]])
    return gradf(x)-tau*scipy.ndimage.convolve(x, matx)
```

Pour choisir le meilleur $\tau$, on va exéctuer l'alogithme de descente de gradient à pas fixe avec différents $\tau = 0.001, 0.01, 0.1, 1$.

```
tauL=[0.001,0.01,0.1,1]
im=[0,0,0,0]
niter = 2000
for i in range(4):
    tau=tauL[i]
    L = 1+8*tau
    t = 1/L
    im[i],b=gradient_descent_fixed_step_size(g,gradg,x0,niter,t,verb=False)
    imshowlistgray([imggray, x0, im[i]],['original', 'init', f'tau={tauL[i]}'])
```

Pout $\tau = 0.001$, on voit que l'image est défloutée mais il y'a toujours un bruit. Pour $\tau = 0.01$ l'image se défloute mais on voit encore un peu de bruit. Pour $\tau = 0.1$ on ne voit plus le bruit le

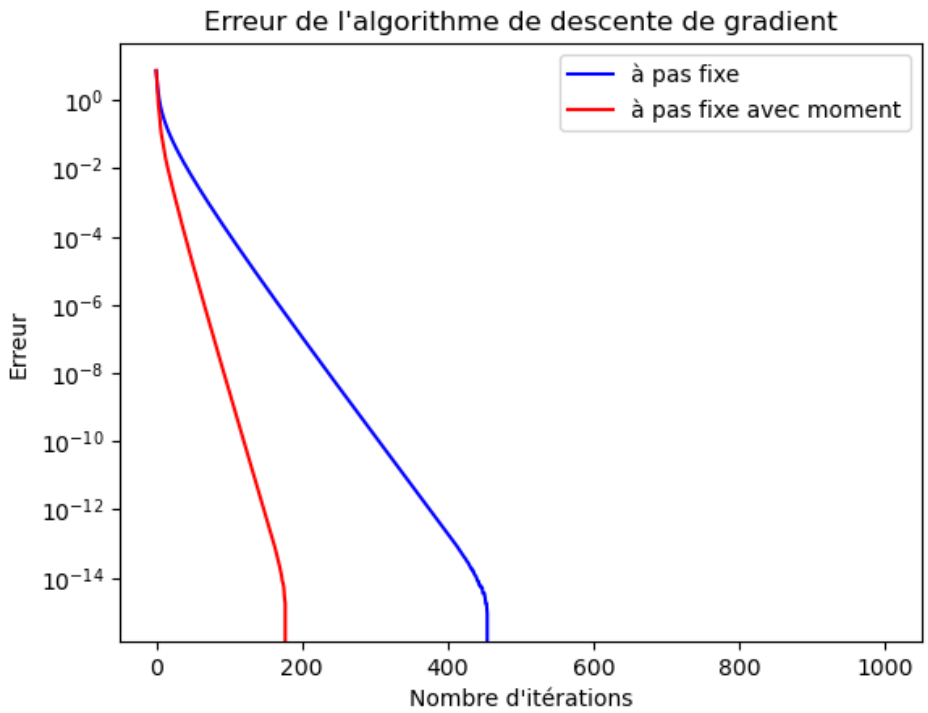
bruit et l'image se défloute mais un peu moins que les deux autres au dessus. Pour $\tau = 1$, il n'y a pas de bruits mais l'image reste flou. Donc $\tau = 0.1$ semble être le meilleur choix.

Comme juste avant, on exécute les deux algorithmes avec cette fonction, on calcul xstar et on compare l'erreur.

```
tau=0.1
L = 1+8*tau
t = 1/L
niter = 1000
alpha=1/L
beta=0.5

x2s, FXk2s = gradient_descent_fixed_step_size(g,gradg,x0,niter,t,verb=False)
x2m, FXk2m = gradient_descent_fixed_step_size_with_moment(g,gradg,x0,niter,alpha,beta,verb=False)

imshowlistgray([imggray, x0, x2s, x2m],['original', 'init', 'pas fixe', 'pas fixe avec moment'])
```

```
tau=0.1
L = 1+8*tau
t = 1/L

Xstar2, FXkstar2 = gradient_descent_fixed_step_size(g,gradg,x0,1000*100,t)
```

```
errFXk2s=[FXk2s[i]-FXkstar2[-1] for i in range(len(FXk2s))]
errFXk2m=[FXk2m[i]-FXkstar2[-1] for i in range(len(FXk2m))]

plt.plot(errFXk2s, color='blue')
plt.plot(errFXk2m, color='red')
plt.yscale('log')

# Titre du graphique
plt.title("Erreur de l'algorithme de descente de gradient")

# Légendes des courbes
plt.legend(['à pas fixe', 'à pas fixe avec moment'])

# Étiquettes des axes
plt.xlabel("Nombre d'itérations")
plt.ylabel("Erreur")

# Afficher le graphique
plt.show()
```




On voit bien que l'image est bien défloutée et que le bruit n'est plus présent. Quand on compare l'erreur pour chaques itérations, on voit que l'erreur de l'algorithme de descente de gradient à pas fixe avec moment est plus petite.

En conclusion, nous avons étudié en détail l'algorithme de descente de gradient avec moment, également connu sous le nom d'algorithme de la boule lourde, et son application dans le domaine de la restauration d'images.

Dans les sections précédentes, nous avons examiné les preuves théoriques de convergence de l'algorithme de descente de gradient avec moment pour différentes classes de fonctions. Nous avons montré que cet algorithme peut accélérer la convergence par rapport à l'algorithme de descente de gradient traditionnel dans certaines conditions, notamment pour les fonctions convexes à gradient Lipschitz et les fonctions fortement convexes.

Ensuite, nous avons réalisé une comparaison numérique entre l'algorithme de descente de gradient avec moment et l'algorithme de descente de gradient standard. Les résultats ont démontré que l'algorithme de descente de gradient avec moment présente généralement une convergence plus rapide et une efficacité supérieure, ce qui en fait une méthode intéressante à utiliser dans les problèmes d'optimisation.

Enfin, nous avons appliqué l'algorithme de descente de gradient avec moment à un problème spécifique de restauration d'images, à savoir le déflouage par régularisation de Tichonov. En utilisant Python, nous avons mis en œuvre l'algorithme pour restaurer des images floues et avons constaté son efficacité dans la récupération des détails et des contours perdus.

En résumé, l'algorithme de descente de gradient avec moment offre une approche prometteuse pour résoudre les problèmes d'optimisation, en particulier dans le domaine du traitement d'images. Son utilisation peut accélérer la convergence et améliorer l'efficacité de l'optimisation, ce qui en fait un outil précieux pour les chercheurs et les praticiens travaillant dans ce domaine.

# Références


[1] M1 MAS : Optimisation, support de cours
Université d'Orléans, Année académique 2022-2023

[2] Andersen, Ang.
*Heavy ball method on convex quadratic problem minimization.*
Mathématique et recherche opérationnelle, UMONS, Belgium.

[3] Vu, Trung.
*Convergence of Heavy-Ball Method and Nesterov's Accelerated Gradient on Quadratic Optimization.*

[4] Ghadimi, Euhanna ; Feyzmahdavian, Hamid Reza ; Johansson, Mikael.
*Global convergence of the Heavy-ball method for convex optimization.*

[5] Notebook : M1MAS_moment_gradient_descent.ipynb